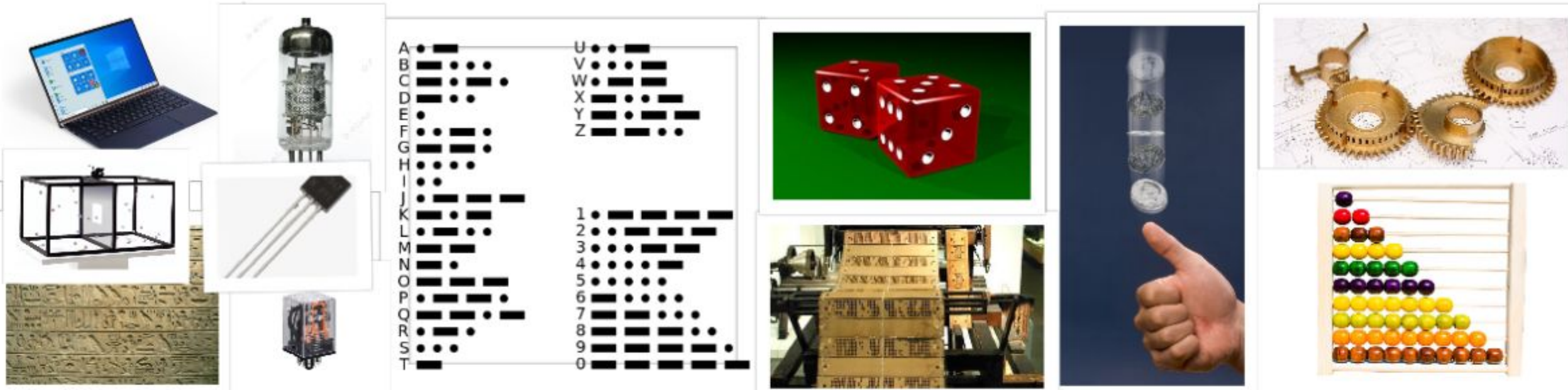




Computer Architecture (CS F342)

Motivation and Introduction
Automatic, Single & General Purpose Computing

Class Schedule

- Google Classroom (Class materials)
- Moodle or Quanta (Labs)

Day	Venue	Time
Tuesday	LT-1	10 AM to 10:50 AM
Wednesday	LT-1	5 PM to 5:50 AM
Thursday	LT-1	10 AM to 10:50 AM
Thursday [Lab.]	CC-219	2 PM to 3:50 PM

Instructors

- Dr Kanchan Manna (kanchanm@goa.bits-pilani.ac.in)

TAs

TAs

Tentative Evaluation Guideline

Components	Duration (Mints)	Weightage (%)	Date	Nature of Component
Regular Labs.	-	20	Lab 1: Aug 6 Lab 2: Aug 13 Lab 3: Aug 20 Lab 4: Aug 27 Lab 5: Sep 03 Lab 5: Sep 10 Lab 6: Sep 17 Lab 7: Sep 24 Lab 9: Oct 1 Lab 10: Oct 15 Lab 11: Oct 22 Lab 12: Oct 29 Lab 13: Nov 5 Lab 14: Nov 12 Lab 15: Nov 19 Lab 16: Nov 26	Open Book Best n-2 of n
Lab. Test	120	15	Lab : Lab :	TBA
Midsem	90	25	As per the timetable	TBA
Comprehensive	180	40	As per the timetable	TBA

Text and Reference Books

Textbooks:

(T1) Computer Organization and Design: The Hardware Software Interface MIPS Edition by David A. Patterson and John L. Hennessy.

(T2) Computer Architecture: A quantitative Approach by David A. Patterson and John L. Hennessy.

Reference Books:

(R1) Digital Design: With a Introduction to the Verilog HDL by M. Morris Mano & Michael D. Ciletti

(R2) Verilog HDL: A Guide to Digital Design and Synthesis by Samir Palnitkar.

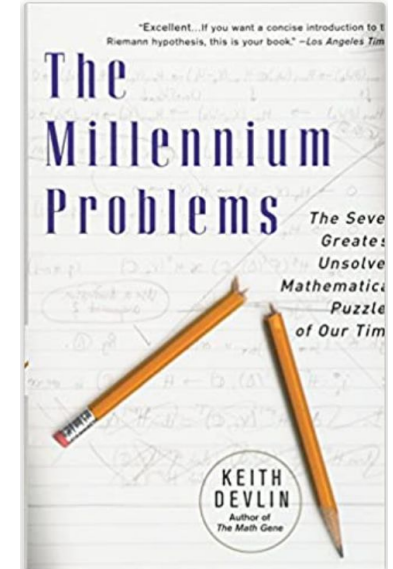
(R3) Computer Organisation & Architecture: Designing for performance by William Stallings.

We shall answer the following questions

1. What's computation?
2. What's uncomputable?
3. What's automatic computation/automation?
4. What's a single-purpose microprocessor?
5. What's a general-purpose microprocessor?
6. How does a laptop solve the problems?
7. Past & Future of Microprocessors

What is the meaning of Computable?

- What is meant to be computable?
 - Is a number prime?
 - An algorithm is present
- Is there anything to uncomputable?
 - Is a number random?
 - Is $P = ? NP$
 - An algorithm is not present **till now**



Unsolved problems: https://en.wikipedia.org/wiki/Millennium_Prize_Problems?fbclid=IwAR1I9mERgOdeaKJASjcXE9AaMht3U6x_4zc0orU_aEC49Vt5U35ZOZB3-Lo
https://en.wikipedia.org/wiki/Smale%27s_problems?fbclid=IwAR3K5n01NUlpeQy_AsXen6dKtMSDzI5RuQ6xOrXulcNRmdlvR_kPiqVnyEQ

© Kanchan Manna; BITS-Pilani, Goa Campus, India.

A problem: Addition of two numbers

- Two numbers: -2 and 8
- Result: 6
- How do humans calculate it?
- How does a calculator/computer/integrated circuit calculate it?
- Is there an algorithm available for this?

A problem: Addition of two numbers

- How does a calculator/computer/IC calculate it?
- Representation of numbers (sign & magnitude)
 - Binary (Digital) system and width
- Is there an algorithm available for this?

Full-Adder (1-bit)

Half-Adder

Input		Output	
A	B	S	Cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S = A \text{ (XOR) } B$$

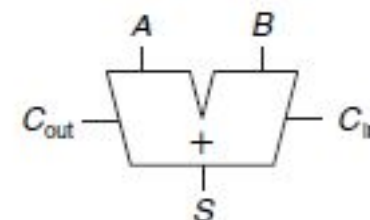
$$C_{out} = A \text{ (AND) } B$$

Input			Output	
Cin	A	B	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Find the relation between input variables and output variables for compressing the table.

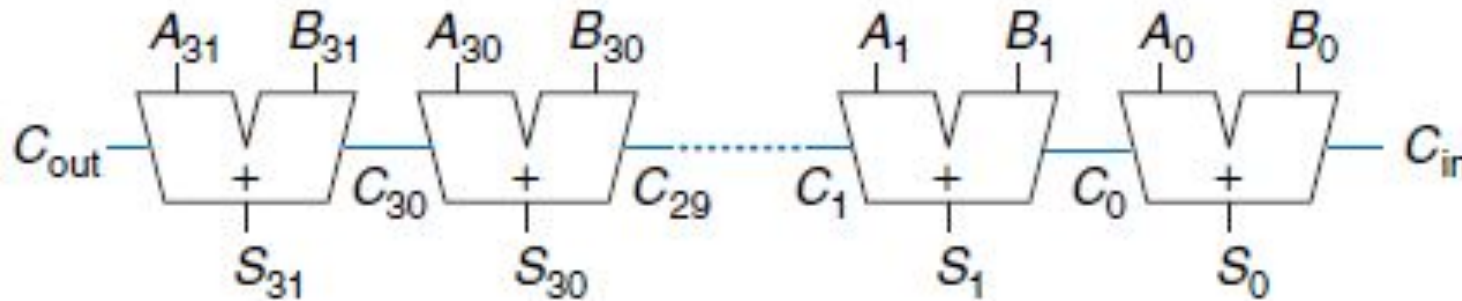
$$S = A \text{ (XOR) } B \text{ (XOR) } C_{in}$$

$$C_{out} = A \text{ (AND) } B + A \text{ (AND) } C_{in} + B \text{ (AND) } C_{in}$$



A problem: Addition of two numbers

- How does a calculator/computer calculate it?
- Is there an algorithm available for this?
 - Ripple-Carry Adder/Algorithm
- Are there efficient algorithms available for this?



- Datapath
- Controller

Can we write a C-code for the above circuit?

Basic Elements: Mapping of High-level Construct onto Digital Construct

Basic Elements: Mapping of High-level Construct onto Digital Construct

How does one represent the data?

For example,
C language provides datatype to represent and store the data.

How is this data stored in digital components such as mobile phones, laptops, smartwatches, etc.?

In the storage elements: Registers and Memory

Basic Elements: Mapping of High-level Construct onto Digital Construct

Register:

32-bit register



Basic Elements:

Mapping of High-level Construct onto Digital Construct

Register:

Each location of the register is made of a **latch** or **flip-flop**

A latch or flip-flop

- RS
- **D**
- T

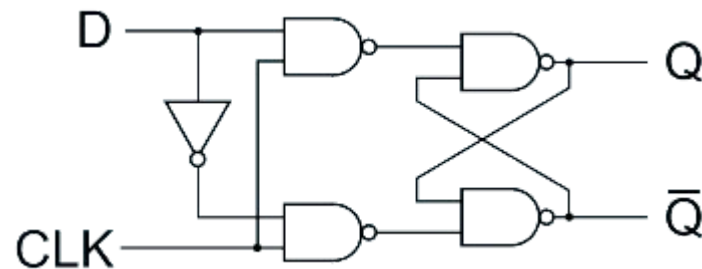


Fig: D-Latch

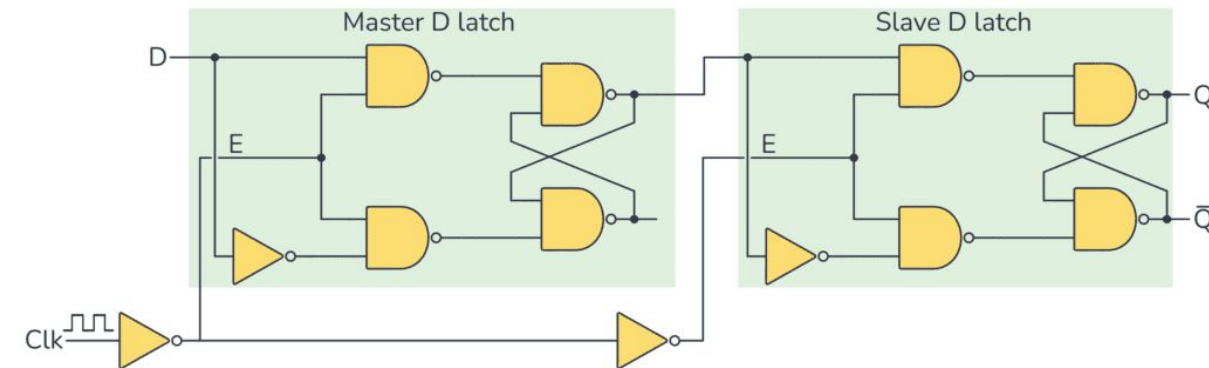
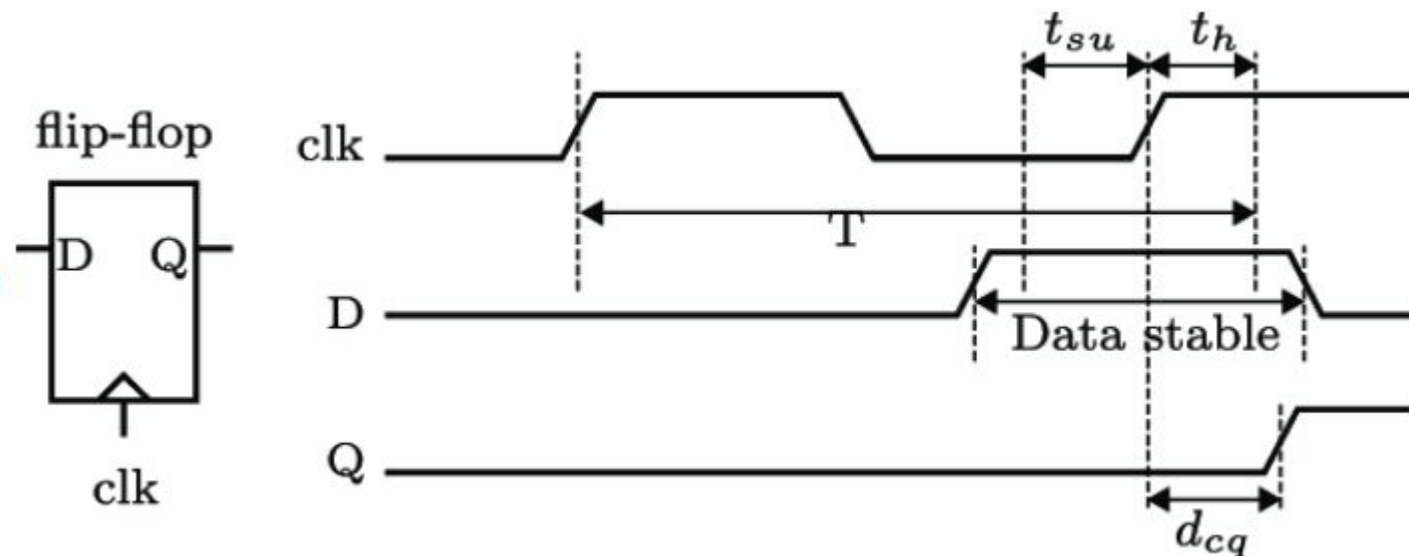


Fig: D-ff

Why do we need a clock in the CKT?

Basic Elements: Clock

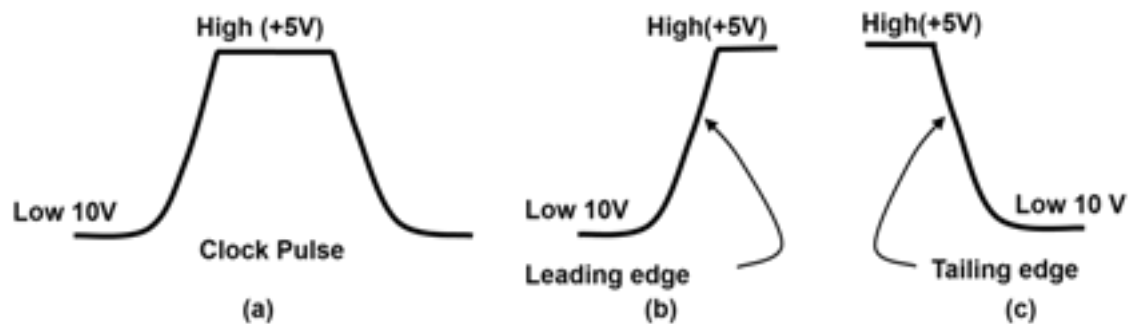
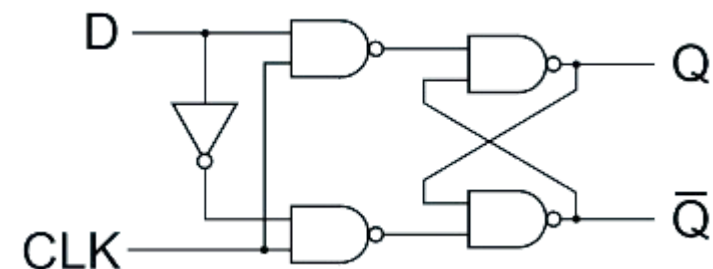


t_{su} : setup time

t_h : Hold time

t_{cq} : Clock-to-Q (q)

T : Clock (clk) Period



+5 volts, logic 1
+10 volts, logic 0

CLK edge sensitive

Basic Elements:

Verilog D-Latch and D-F/F

```
module d_latch (  input d,          // 1-bit input pin for data
                  input en,          // 1-bit input pin for enabling the latch
                  input rstn,        // 1-bit input pin for active-low reset
                  output reg q);     // 1-bit output pin for data output

// This always block is "always" triggered whenever en/rstn/d changes
// If reset is asserted then output will be zero
// Else as long as enable is high, output q follows input d
always @ (en or rstn or d)
    if (!rstn)
        q <= 0;
    else
        if (en)
            q <= d;
endmodule
```

```
module dff(clk,d,q);
input clk,d;
output reg q;

always @ (posedge clk)begin
    q <= d;
end

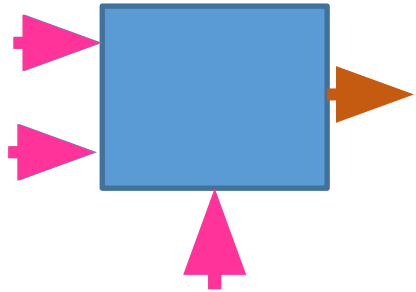
endmodule
```

How to map Algorithm's elements onto Architectural elements

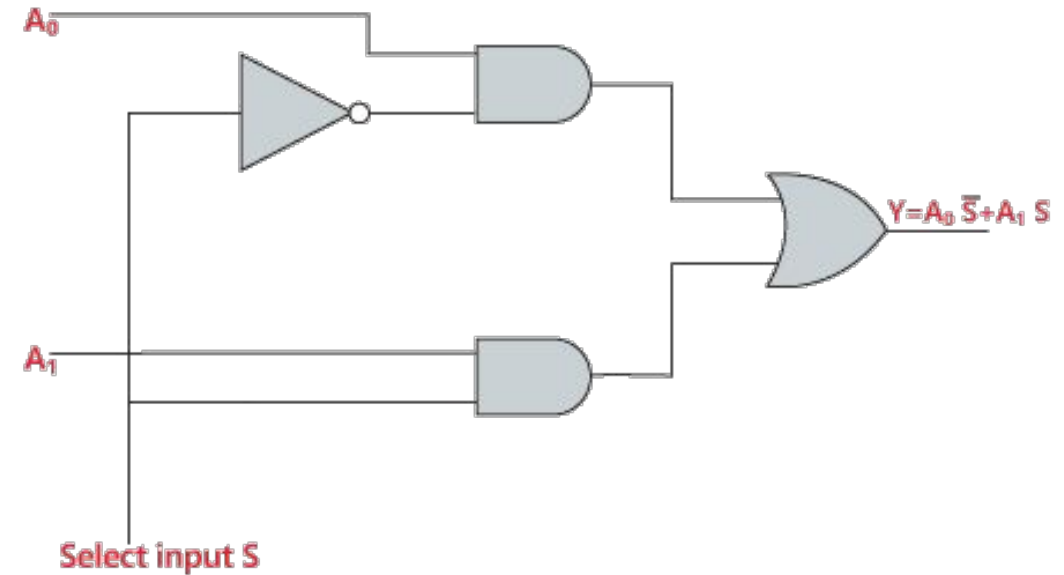
High-level Construct	Digital Construct	Type of Component
Scalars/variable	Registers or wires	Sequential if Reg.
Arrays	Memories	Sequential
Operators (+,-,*,/, etc)	Functional unit	Combinational
Control flows [if-else, switch & loop]	Control unit	Combinational/Sequential

How to map Algorithm's elements onto Architectural elements

```
if (sel)
    a = 10;
else
    a = 5;
```



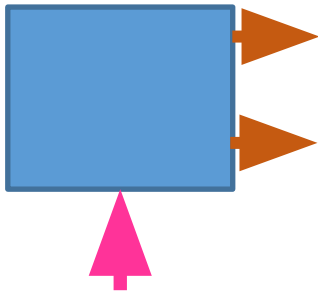
Multiplexer



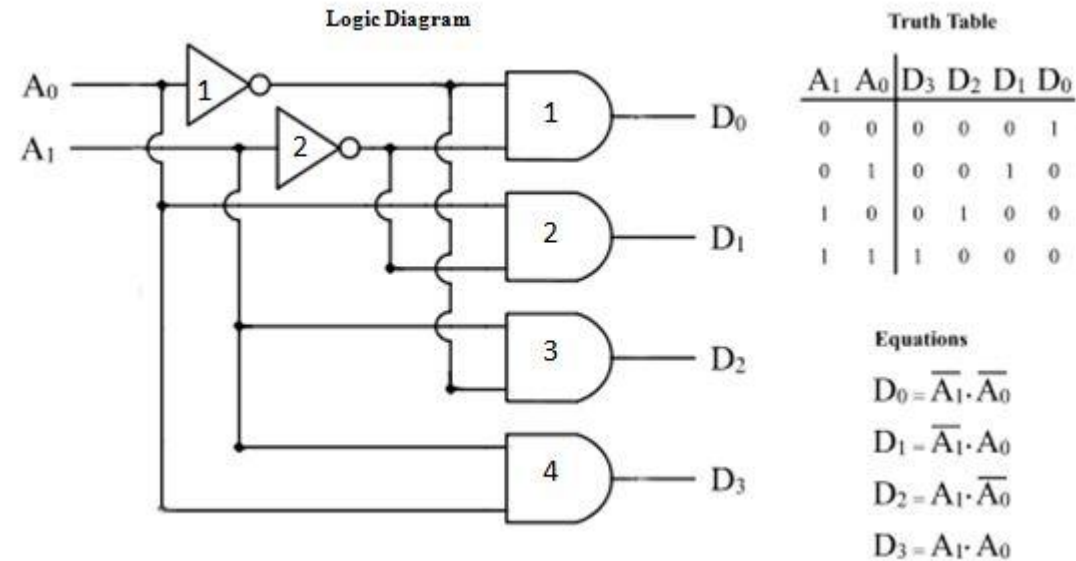
How to map Algorithm's elements onto Architectural elements

```

if (sel)
    a = 10;
else
    b = 5;
  
```



Decoder

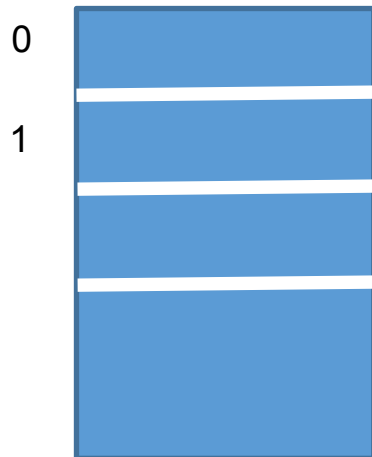


How to map Algorithm's elements onto Architectural elements

```
int Mem[1024], Data;
```

```
Mem [10] = Data;
```

```
Data = Mem [20];
```



Mem

Write operation

```
switch (Addrs){
    case LOC0:
        Mem[LOC0] = Data;
        break;
    case LOC1:
        Mem[LOC1] = Data;
        break;
    case LOC2:
        Mem[LOC2] = Data;
        break;
    case LOC3:
        Mem[LOC3] = Data;
        break;
}
```

Decoder

Read operation

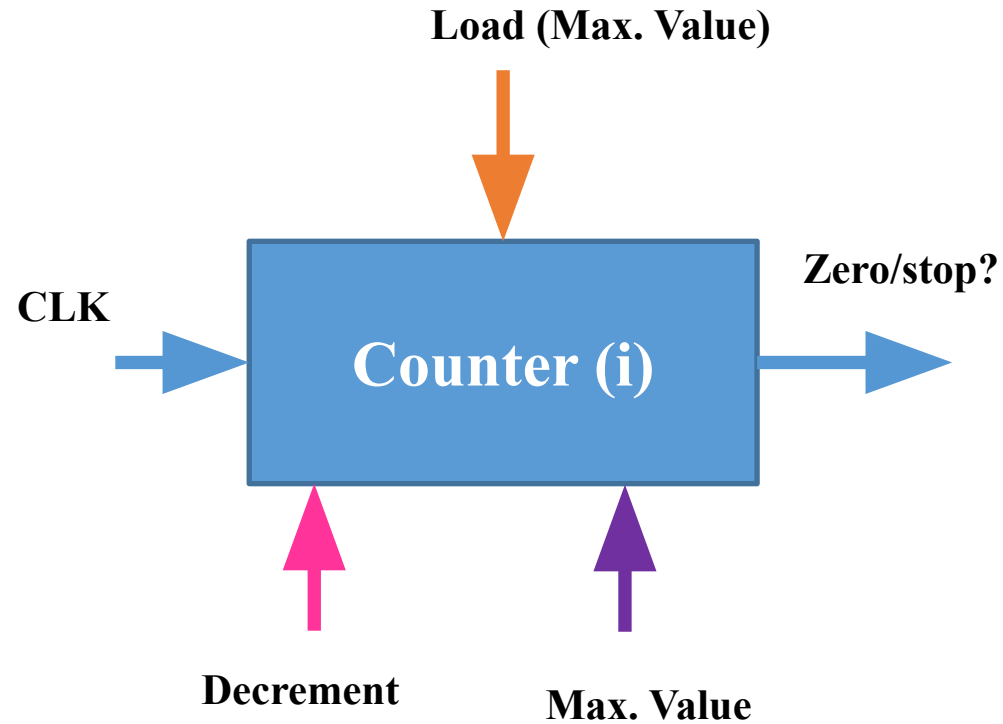
```
switch (Addrs){
    case LOC0:
        Data_out = Mem[LOC0]
        break;
    case LOC1:
        Data_out = Mem[LOC1]
        break;
    case LOC2:
        Data_out = Mem[LOC2]
        break;
    case LOC3:
        Data_out = Mem[LOC3]
        break;
}
```

Multiplexer

How to map Algorithm's elements onto Architectural elements

```
int i;  
  
for (i=10; i>0; i--){  
  
}
```

Counter/Register



Counter Design Using D-F/F:

https://www.youtube.com/watch?v=ts4g_NUuHAc

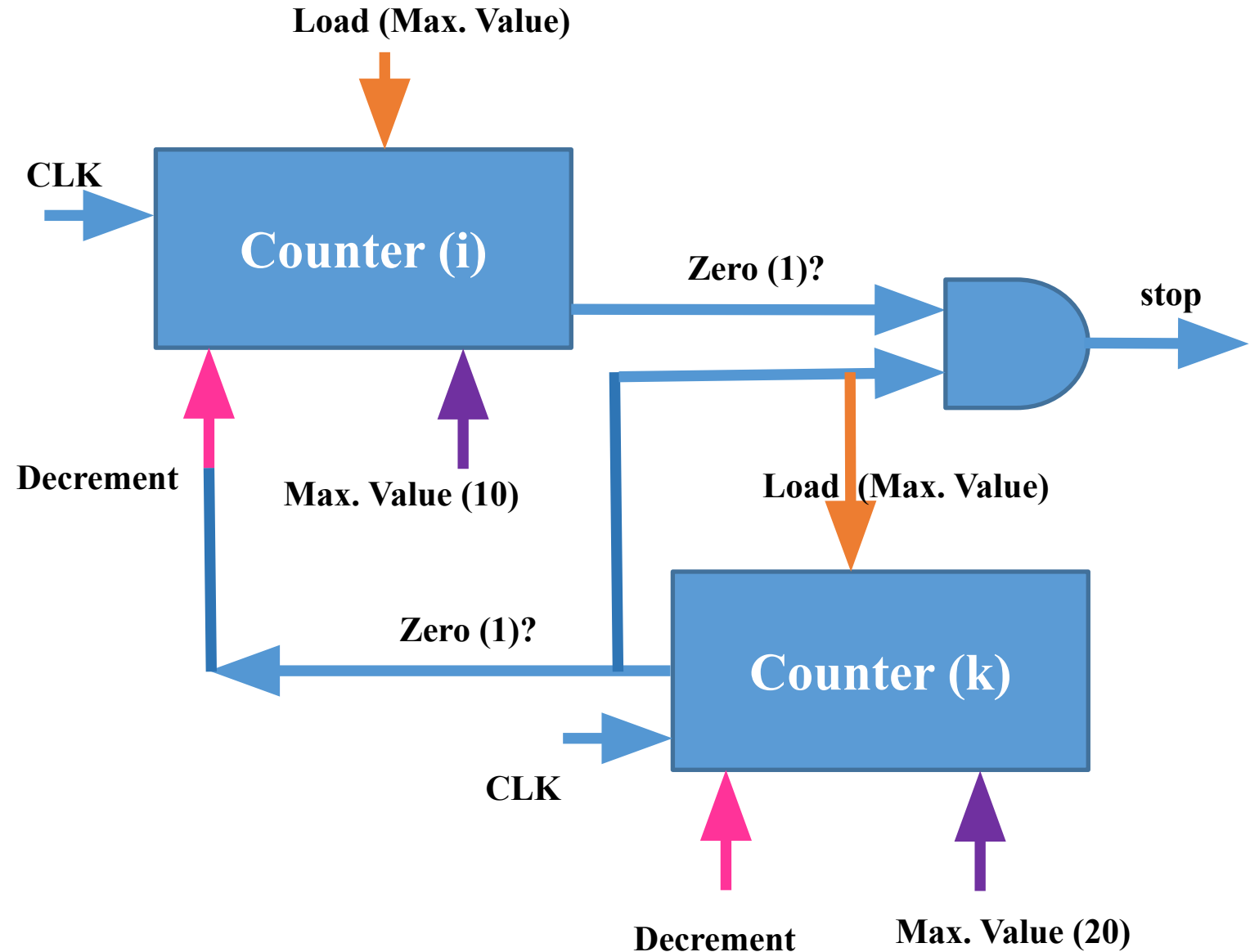
How to map Algorithm's elements onto Architectural elements

```
int i, k;
```

```
for (i=10; i>0; i--){
  for (k=20; k>0; k--){
  }
}
```

Zero = 1 if the content of the counter is 0.

Stop = 1 if both counters value is 0.

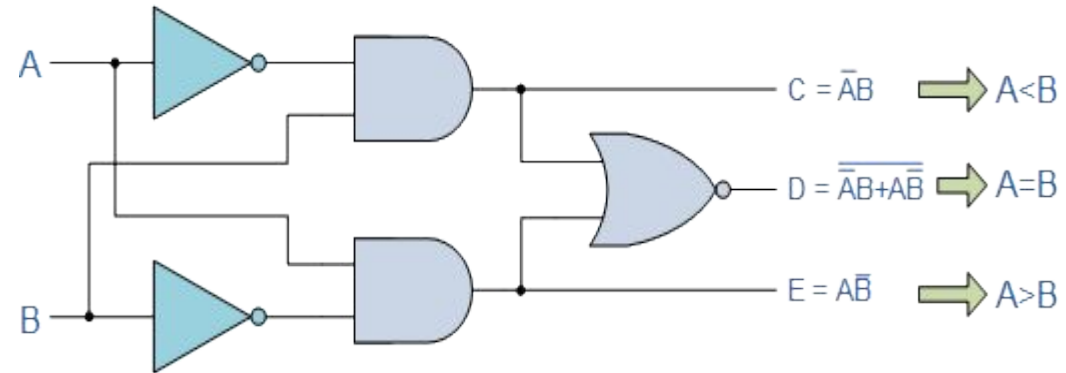
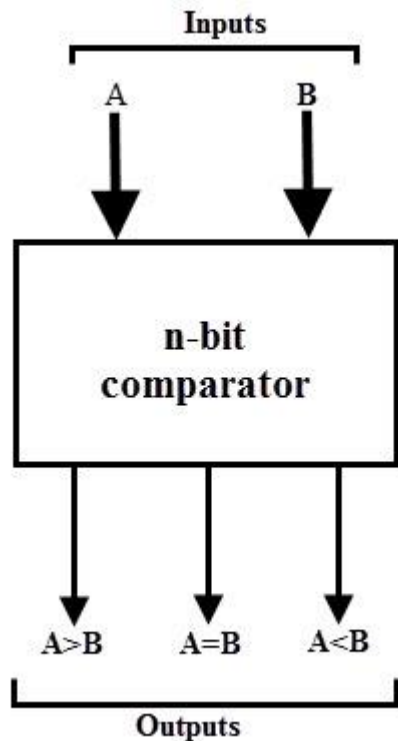


Counter Design Using D-F/F:

https://www.youtube.com/watch?v=ts4g_NUuHAc

How to map Algorithm's elements onto Architectural elements

Comparison



A problem: Find the minimum and maximum number from a set of numbers

- Set of nos: 1, 2, -8, 0, 23, 11, -10
- Min = -10 and Max = 23
- Is there an algorithm to solve it automatically?

Input: set of n nos stored in A

Output: Min & Max

Min = $-\infty$

Max = ∞

Do scan i-th no.

 If Max < A[i] then

 Max = A[i]

 If Min > A[i] then

 Min = A[i]

Until i reaches to n

Stop

A problem: Find the minimum and maximum number from a set of numbers

- Is there an algorithm to solve it automatically?
- Yes
- Is there an architecture for that algorithm?

Input: set of n nos stored in A

Output: Min & Max

Min = ∞

Max = $-\infty$

Do scan i -th no.

 If Max < $A[i]$ then

 Max = $A[i]$

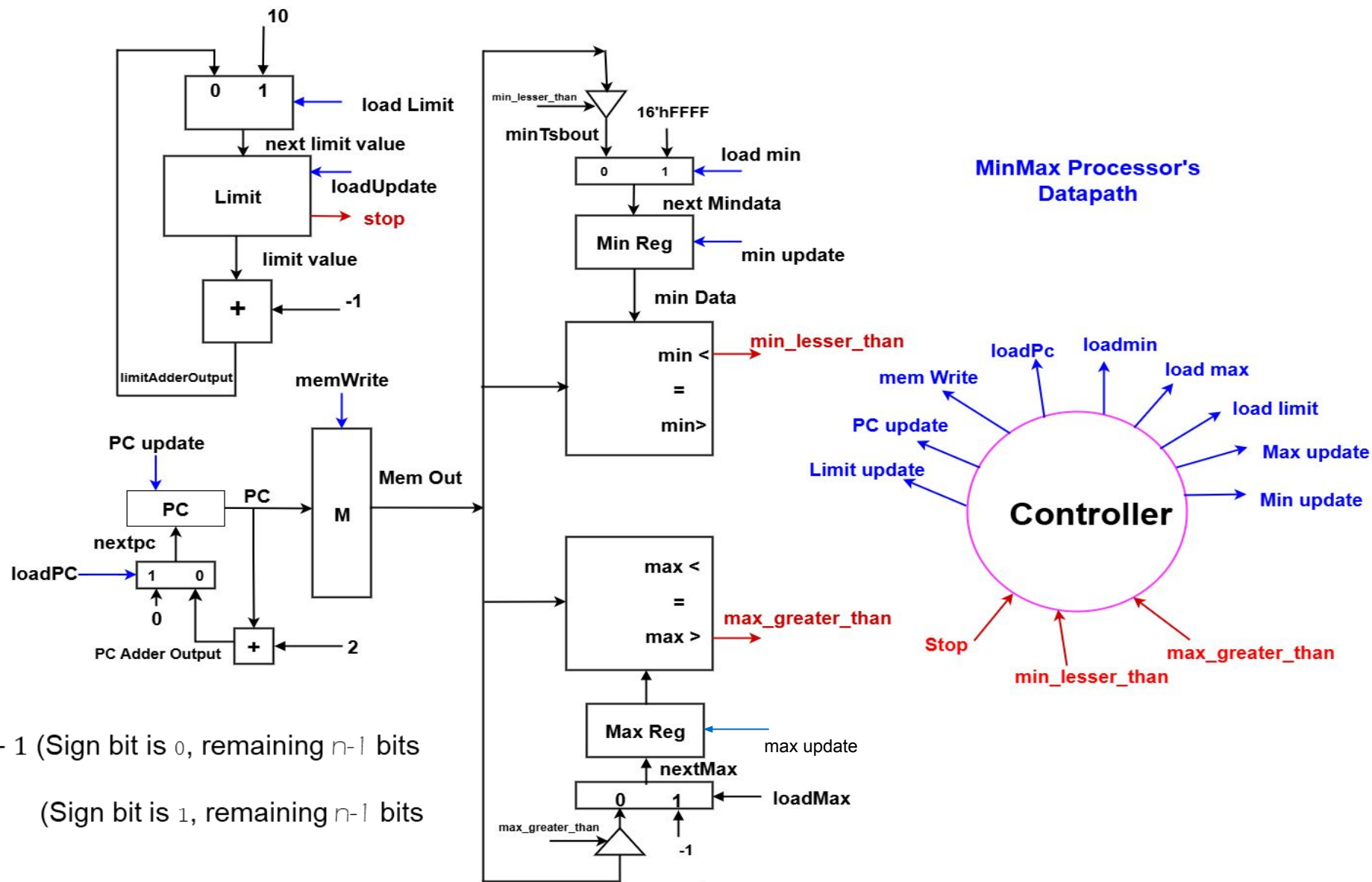
 If Min > $A[i]$ then

 Min = $A[i]$

Until i reaches to n

Stop

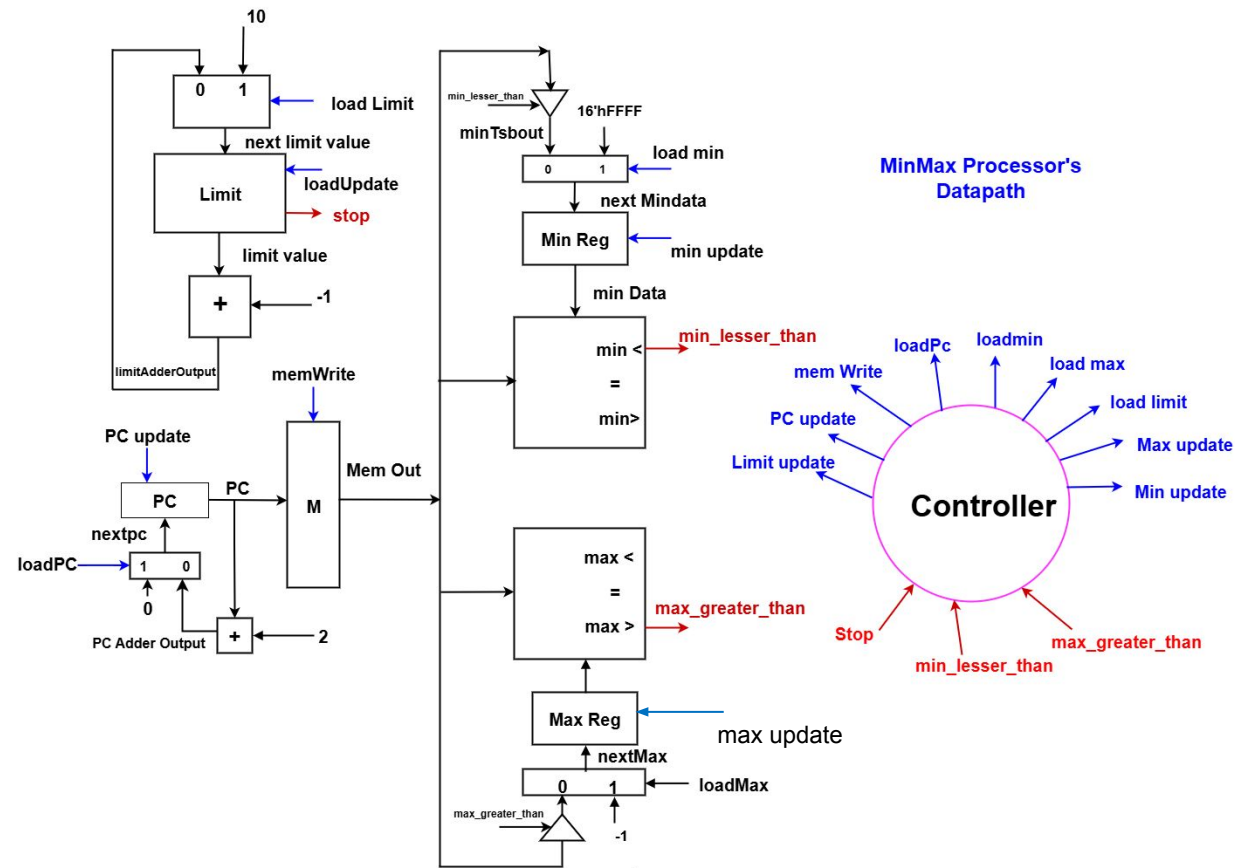
MinMax Processor



• **Maximum Value:** $2^{n-1} - 1$ (Sign bit is 0, remaining $n-1$ bits are 1)

• **Minimum Value:** -2^{n-1} (Sign bit is 1, remaining $n-1$ bits are 0)

MinMax Processor



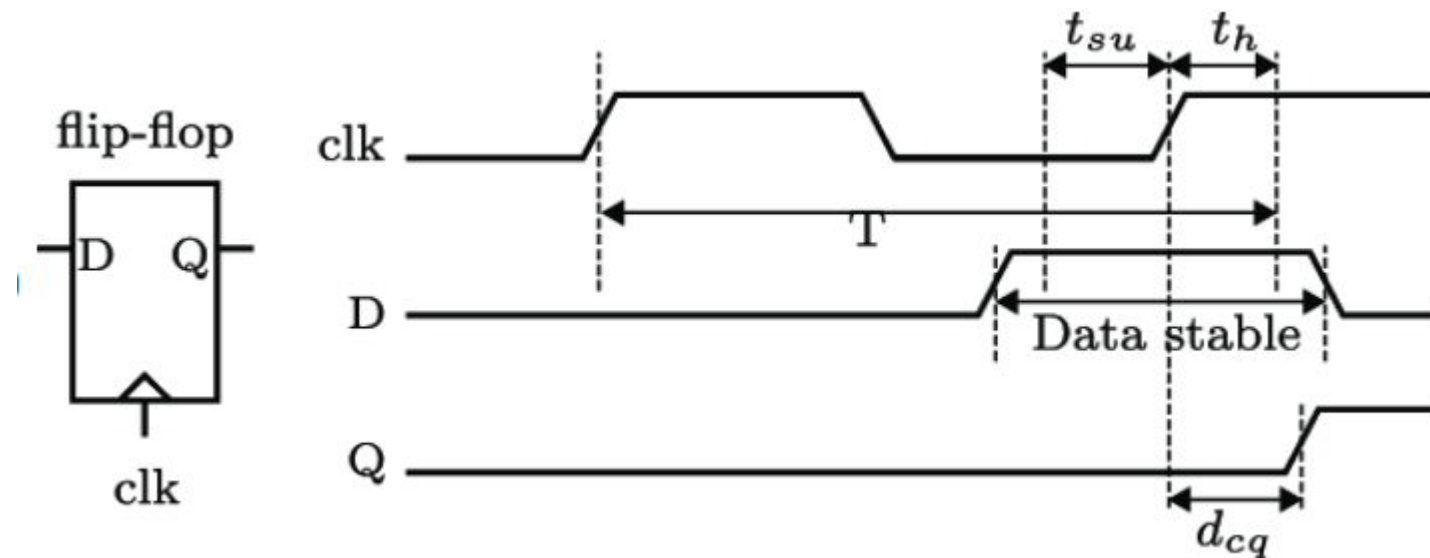
How does one measure the performance of this processor?

After giving inputs to it and time to get the output.

How does one include time in the processor?

Clock

Basic Elements: Clock

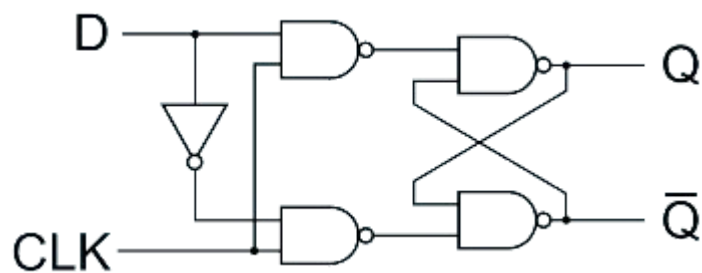


t_{su} : setup time

t_h : Hold time

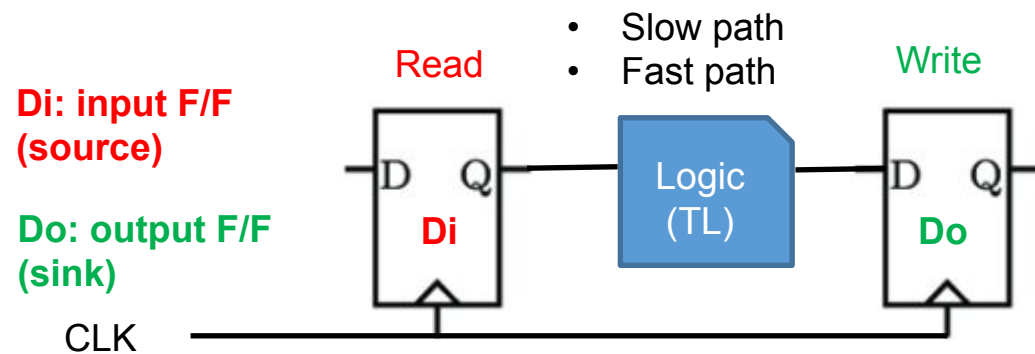
t_{cq} : Clock-to-Q (q)

T : Clock (clk) Period



Setup and Hold time come from the Library.

Data Setup time, Data Hold time, Clock Period calculation



t_{su} : Setup time t_h : Hold time

t_{cq} : Clock-to-Q (q)

T : Clock (clk) Period

TL : Time to compute Logic

Calculate all such input and output pairs of D-ff or Regs. and maintain eq. 1 and eq. 2. Find all T_i

Max (T_i) would be the clock period (T)

Data launched at one flop (**Di**) should be captured at the next flop (**Do**) in the **next** clock edge. Data launched at one edge should be captured at the **next active** clock edge.

Let's consider the clock period as **T**

Data setup time of **Do** (t_{su_Do}) f/f must be

Clock period - t_{su_Do}

Clock period - $t_{su_Do} >$ Data arrival time at **Do**

$$T - t_{su_Do} > t_{cq_Di} + TL$$

$$T > t_{cq_Di} + TL + t_{su_Do} \quad (1)$$

Hold the data in **Do** f/f so that **race conditions** won't happen

Data hold time of in f/f **Do** (t_{h_Do}) must be

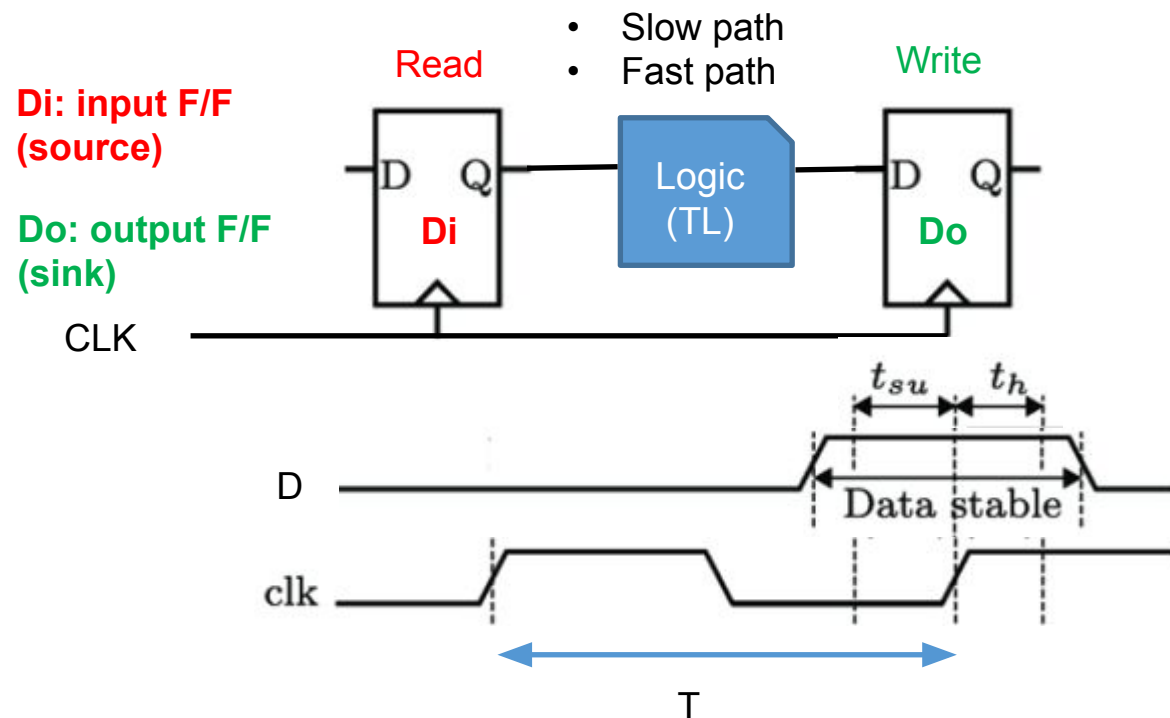
$$t_{cq_Di} + TL > t_{h_Do} \quad (2)$$

What if the previous data's hold time is t_{h_Do} is more than $t_{cq_Di} + TL$?

Clk is also a high-state at that time

Next data would be written in the previous clock

Data Setup time, Data Hold time, Clock period calculation



Let's consider the clock period as T

Data setup time of Do (t_{su_Do}) f/f must be

Clock period (T) - t_{su_Do}

Clock period (T) - $t_{su_Do} >$ Data arrival time at Do

$$T - t_{su_Do} > t_{cq_Di} + TL$$

$$T > t_{cq_Di} + TL + t_{su_Do} \quad (1)$$

Hold the data in Do f/f so that **race conditions** won't happen

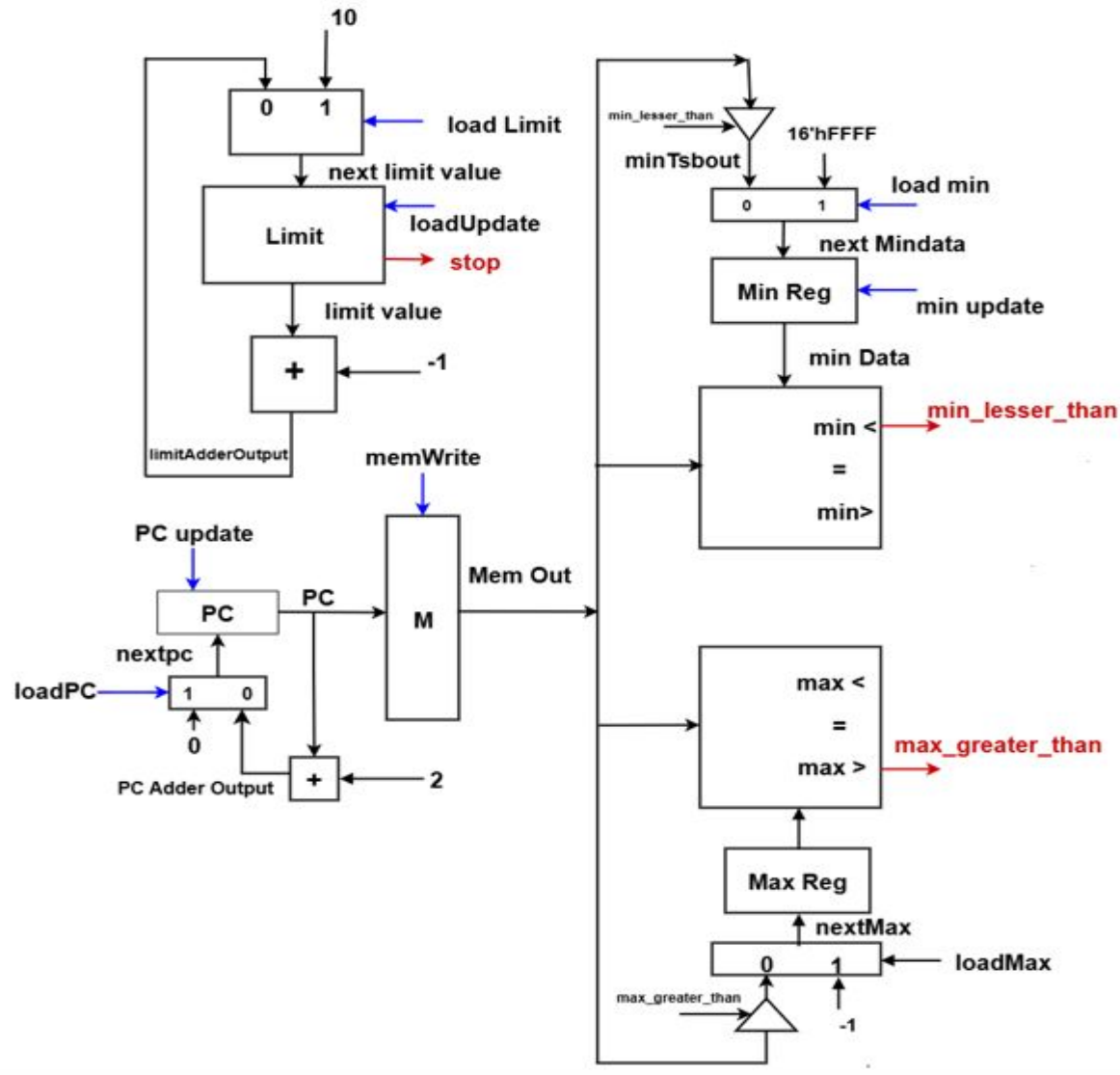
Data hold time of in f/f Do (t_{h_Do}) must be

$$t_{cq_Di} + TL > t_{h_Do} \quad (2)$$

What if the previous data's hold time is t_{h_Do} ?

If data hold time check is violated, data intended to be captured at the next edge will get captured at the same edge.

A problem: Find the minimum and maximum number from a set of numbers



Find clock period (T)

Time (Input_Reg to Output_Reg)

Time (Reg. PC/I to MaxReg)

Time (Reg. PC/i to MinReg)

Time (Reg. limit to limit)?

?

Find maximum Time from all possible Time values

Can we execute other algorithms on this processor?

Special-purpose/Dedicated Processor MinMax Processor

Input: set of n nos stored in A

Output: Min & Max

Min = ∞

Max = - ∞

Do scan i-th no.

If Max < A [i] then

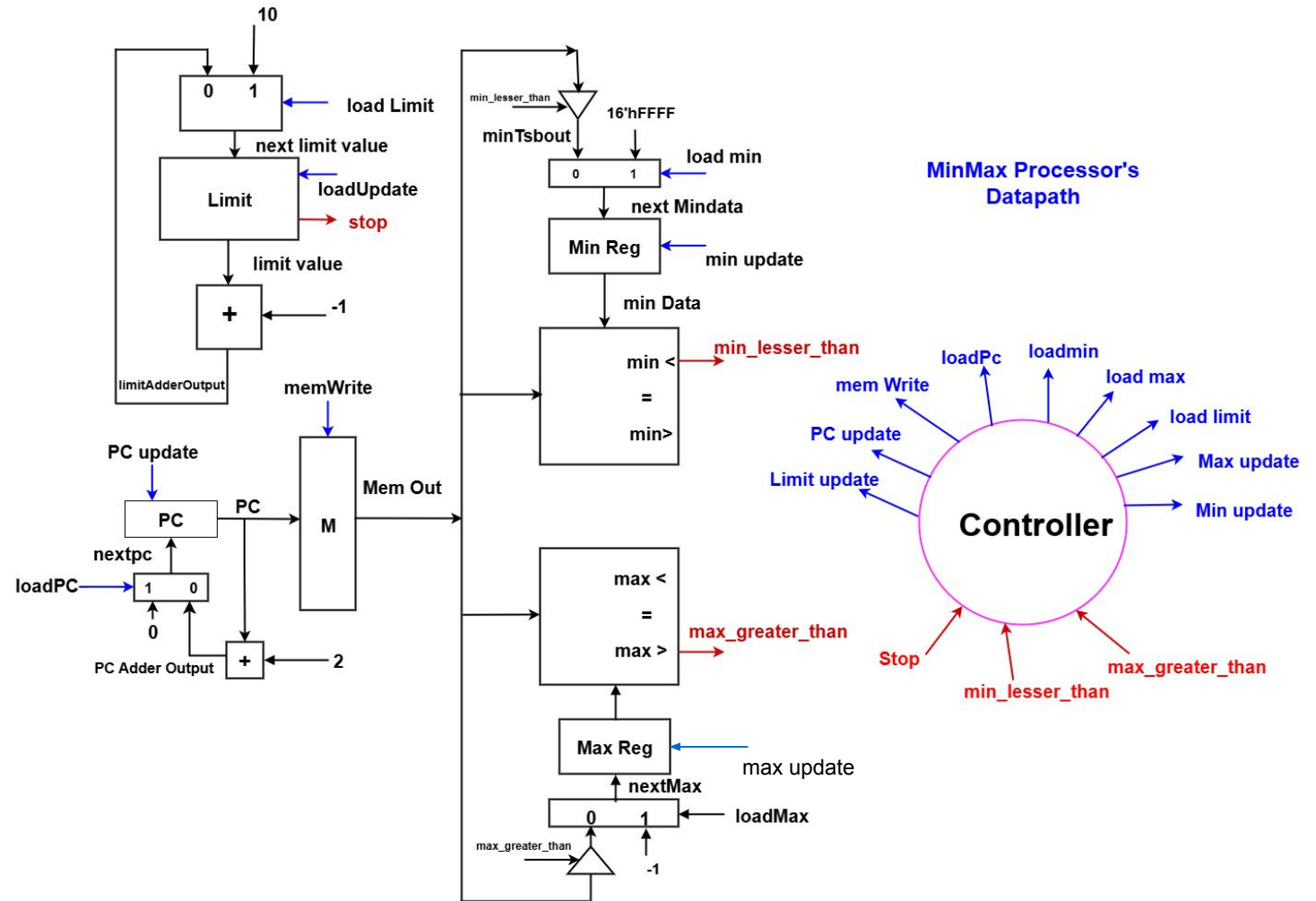
Max = A [i]

If Min > A [i] then

Min = A [i]

Until i reaches to n

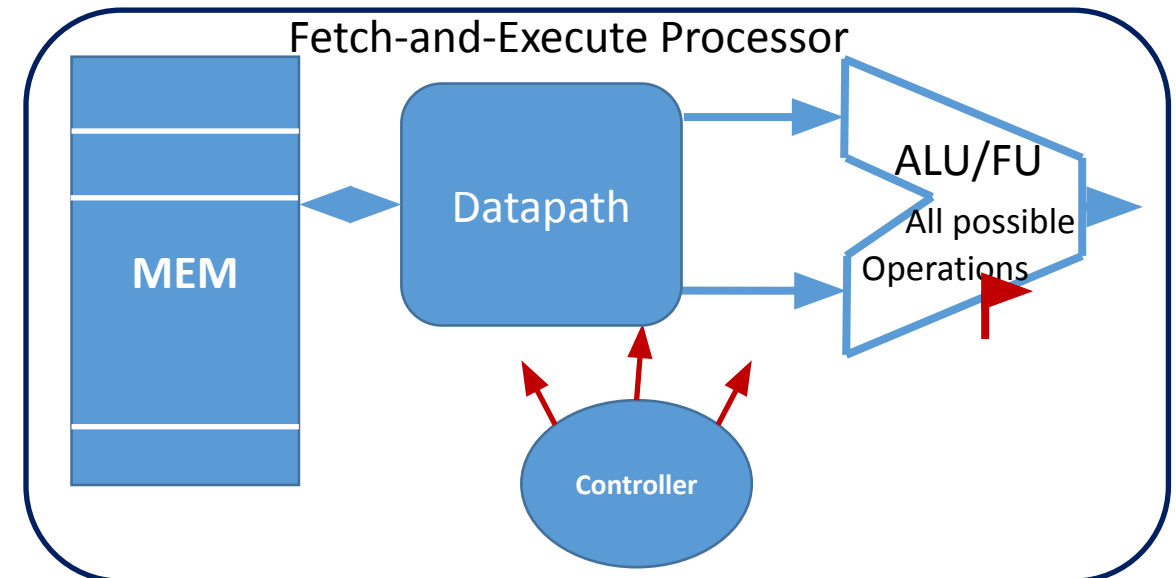
Stop



MinMax Processor

Necessity of General-purpose processor

- Is there an Algorithm which will execute or simulate other Algorithms?
 - The processor executes any algorithms
 - Programmable
 - Turing Model
 - Is there any **limitation** of such an Algorithm?
 - **Halting problem:** Can we have an Algorithm which takes other Algorithm as input and decides that whether given input Algorithm will halt/stop or not, **in general**?
 - Consider [*] such Algo. exists $A(P, D)$. Another Algo. $B(X)$: loop-forever if $A(X, X) = \text{"Halt"}$ else Halt. Next use $B(B)$, it is unable to decide the answer. $A(P, I)$ **doesn't exist**.
 - Used **Self-referential** structure
- What kind of Algorithm do we need for making the processor general purpose or programmable?
 - *Fetch-and-Execute* Algorithm
 - **Stored program** (?) [*]
 - Generalized Datapaths
 - Generalized Functional Unit
 - Proposed by Jhon von Neumann [*]



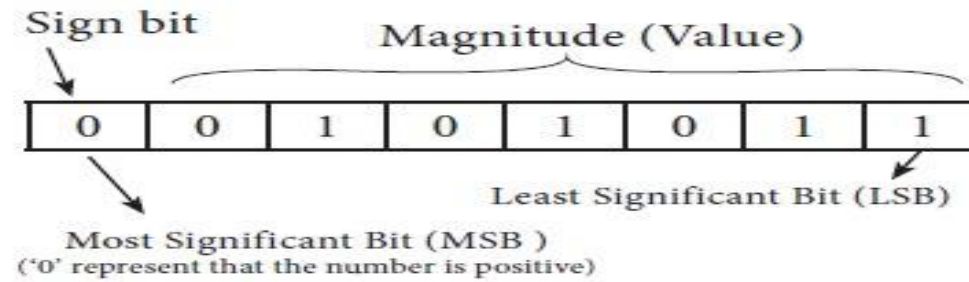
[*] An URL is embedded.

Fetch-and-Execute Algorithm

- What is to be fetched?
 - **Program**/instructions (Birth of the program or software)
 - Representation of instruction (opcode & operand)
 - Representation of number (sign and magnitude) [parallel reasoning]
- Where are the **instructions** coming from?
 - It's **stored** in storage/memory
- How do the instructions execute?
 - What kind of operation is it?
 - Decode the instruction
 - Execute the instruction

Fetch-and-Execute Algorithm

- How does one represent the instruction?
- Is there any similar problem we solved?



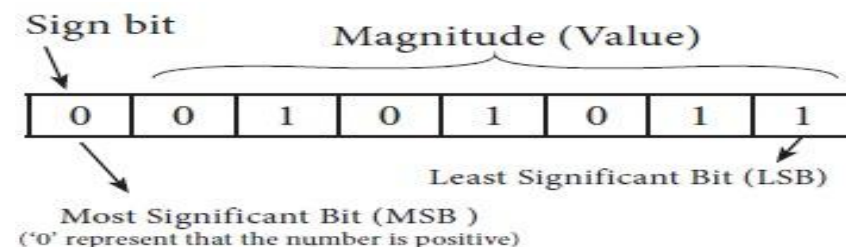
Number representation

- Representation of instruction
 - Instruction format

Fetch-and-Execute Algorithm

Number representation

- Representation of instruction
 - Instruction format



Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

Instruction	Format	6-bit OP	5-bit RS	5-bit RT	5-bit RD	5-bit Shamt	6-bit Funct	Address
add	R	0	reg	reg	reg	0	32	NA
sub	R	0	reg	reg	reg	0	34	NA
addi	I	8	reg	reg	NA	NA	NA	Constant
lw	I	35	reg	reg	NA	NA	NA	Offset
Sw	I	43	reg	reg	NA	NA	NA	Offset
bne	I	5	reg	reg	NA	NA	NA	Offset

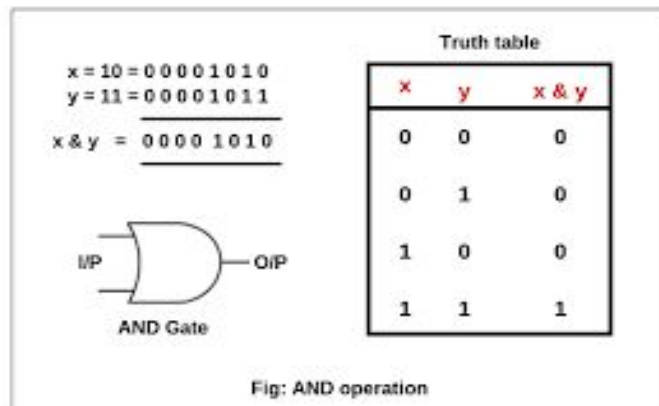
Microprocessor
without
Interlocked
Pipelined
Stages

Fetch-and-Execute Algorithm

Inst. opcode	Opcode
LW	100010
SW	100011
BEQ	000100
R-type	000000
addi	001000
j	000010

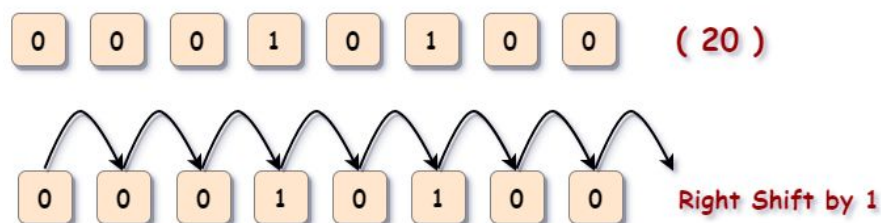
Fetch-and-Execute Algorithm

- Bit wise and (&) and shift operations (<< and >>)



BITWISE OPERATORS		
<u><< Shift Left</u>		
SYNTAX	BINARY FORM	VALUE
$x = 7;$	00000111	7
$x = x \ll 1;$	00001110	14
$x = x \ll 3;$	01110000	112
$x = x \ll 2;$	11000000	192

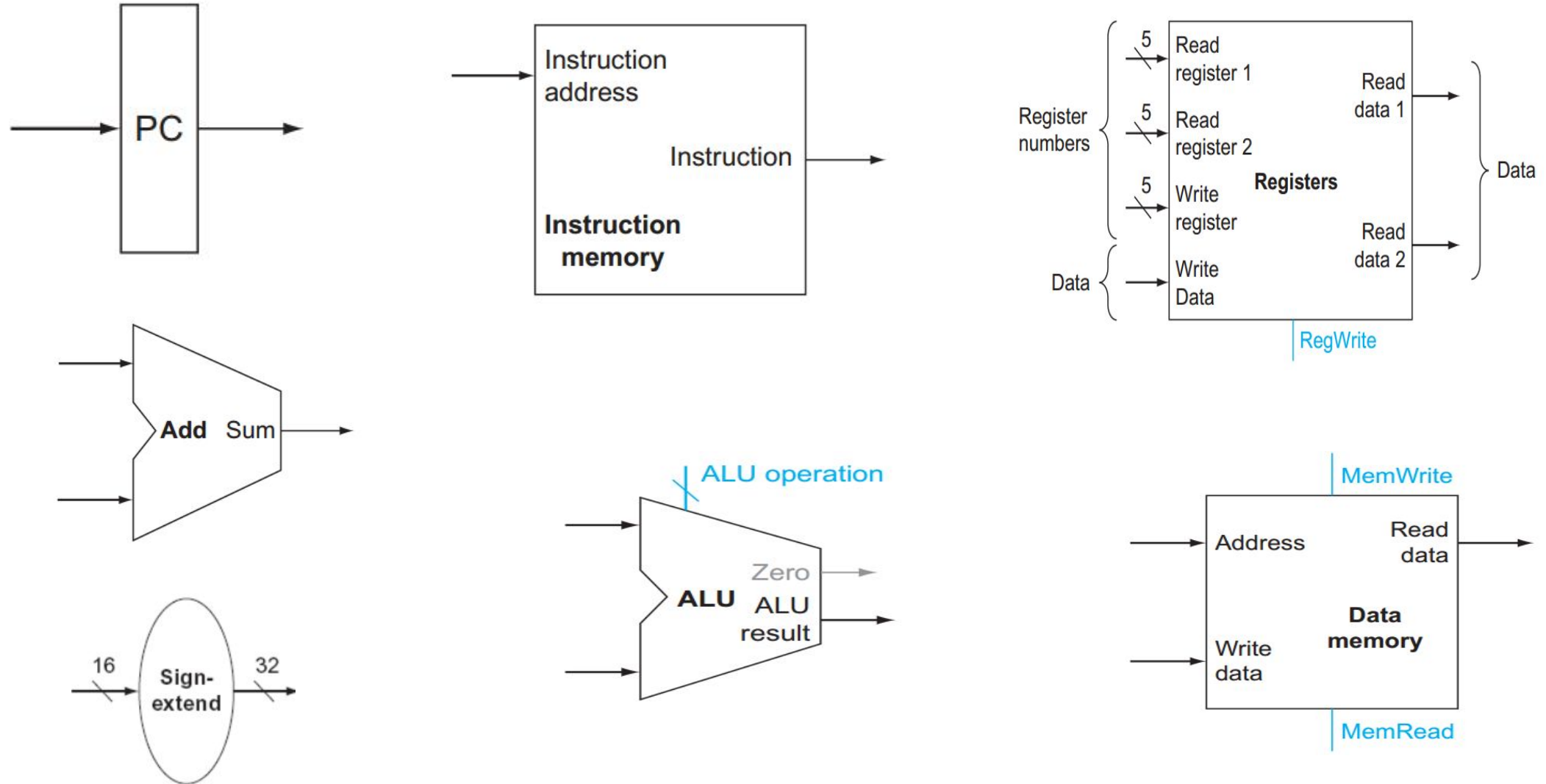
Bitwise Right Shift Operator



© SillyCodes.com

20 >> 1 0 0 0 0 1 0 1 0 (10)

MIPS Processor/Generalized components



Fetch-and-Execute Algorithm

```
#define OP CODE 0b11111100000000000000000000000000
#define RS      0b00000001111100000000000000000000
#define RT      0b00000000000001111100000000000000
#define DST      0b00000000000000000111110000000000 //RD
#define OFFSET  0b00000000000000000011111111111111
```

```
int PC, IMM[1024], DMM[1024], RF[32];
```

```
Load(IMM, DMM);
```

```
Set PC with address 1st instruction which is stored in IMM;
```

```
while (1){
```

```
    switch((IMM[PC] & OP CODE) >> 26){
```

```
        case R-type:
```

```
        RF[(IMM[PC] & DST) >> 11] = ALU(RF[(IMM[PC] & RS) >> 21], RF[(IMM[PC] & RT) >> 16]); PC = PC + 4; break;
```

```
        case SW:
```

```
        DMM[ALU((IMM[PC] & RS) >> 21, (IMM[PC] & OFFSET) )] = RF[(IMM[PC] & RT) >> 16]; PC = PC + 4; break;
```

```
        case LW:
```

```
        RF[(IMM[PC] & RT) >> 16] = DMM[ALU((IMM[PC] & RS) >> 21, IMM[PC] & OFFSET) ]; PC = PC + 4; break;
```

```
        case BNE:
```

```
        IF (ZERO) PC = (PC + 4) + ((IMM[PC] & OFFSET) << 2); ELSE PC = PC + 4; break;
```

```
    }
```

```
}
```

Let's define ALU()

Can we have an Architecture for this Algorithm?

Fetch-and-Execute Algorithm for MIPS Microprocessor (32 bit)

```
#define OPCODE 0b11111100000000000000000000000000
#define RS      0b00000001111100000000000000000000
#define RT      0b00000000000011111000000000000000
#define RD      0b00000000000000000111110000000000
#define SHIFT   0b0000000000000000000000011111000000
#define OFFSET  0b00000000000000000111111111111111
```

```
int PC, IMM[1024], DMM[1024], RF[32], ALUControl; bool ZERO;
```

```
Load(IMM, DMM);
```

```
Set PC with address 1st instruction which is stored in IMM;
```

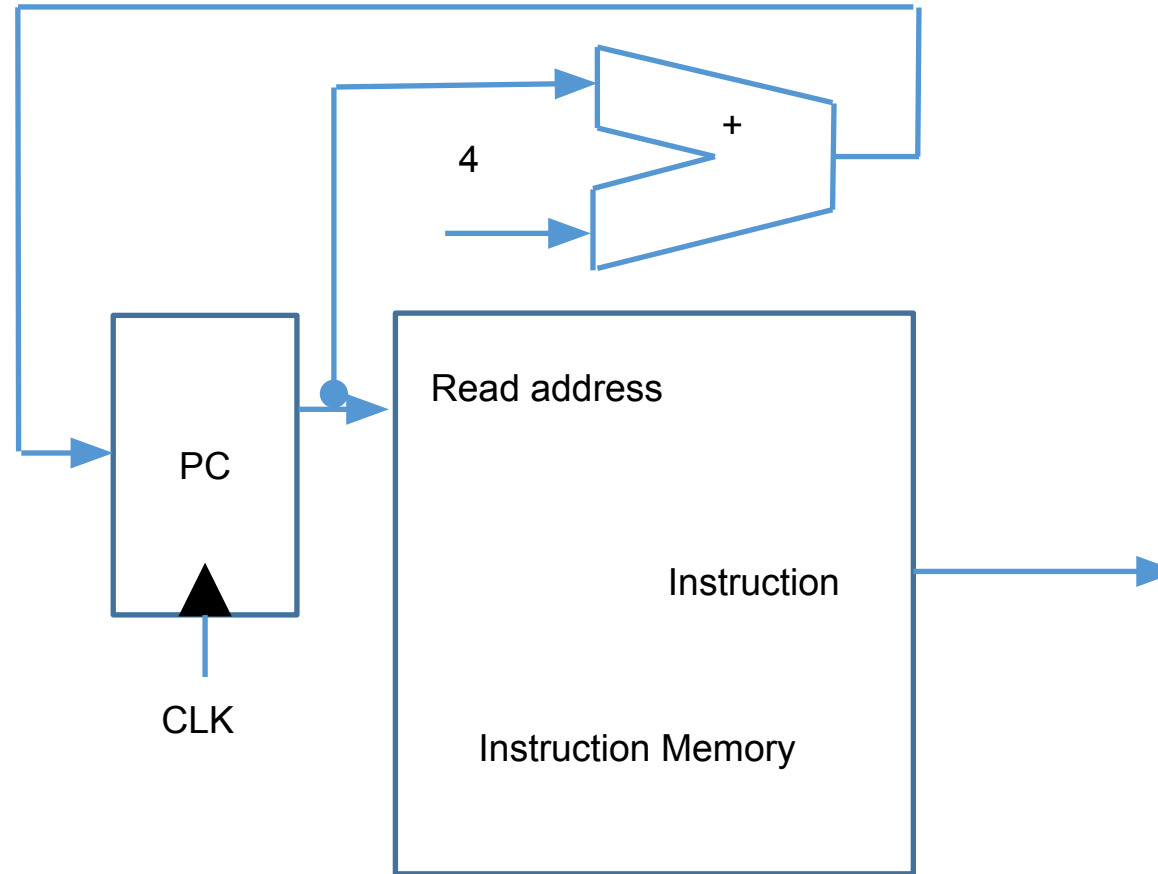
```
while (1){
    switch((IMM[PC] & OPCODE) >>26){
        case R-type:
            Set ALUControl; //ADD, SUB, AND, OR, etc
            RF[IMM[PC] & DST] = ALU(RF[(IMM[PC] & RS)>>21], RF[(IMM[PC] & RT) >>16]); PC = PC + 4;
        case SW:
            Set ALUControl = 0b0010;
            DMM[ALU((IMM[PC] & RS)>>21, (IMM[PC] & OFFSET) )] = RF[(IMM[PC] & RT)>>16]; PC = PC + 4;
        case LW:
            Set ALUControl = 0b0010;
            RF[IMM[PC] & RT]= DMM[ALU((IMM[PC] & RS) >>21, IMM[PC] & OFFSET) ]; PC = PC + 4;
        case BNE:
            Set ALUControl = 0b0110;
            ALU(RF[(IMM[PC] & RS)>>21], RF[(IMM[PC] & RT) >>16]);
            IF (ZERO ==1) PC = (PC + 4) + ((IMM[PC] & OFFSET) <<2); ELSE PC = PC + 4;
    }
}
```

```
ALU(Src1, Src2){
    switch (ALUControl){
        case B-type: ZERO = (Src1 - Src2) == 0 ? 1 : 0;
            break;
        case ADD: return (Src1 + Src2); break;
        ...
    }
}
```

Need conversion for
16 bit offset to 32 bits

Analysis of data-path for **Fetch** stage

Why 4?



Clock Period ($T_{pc-to-pc}$)

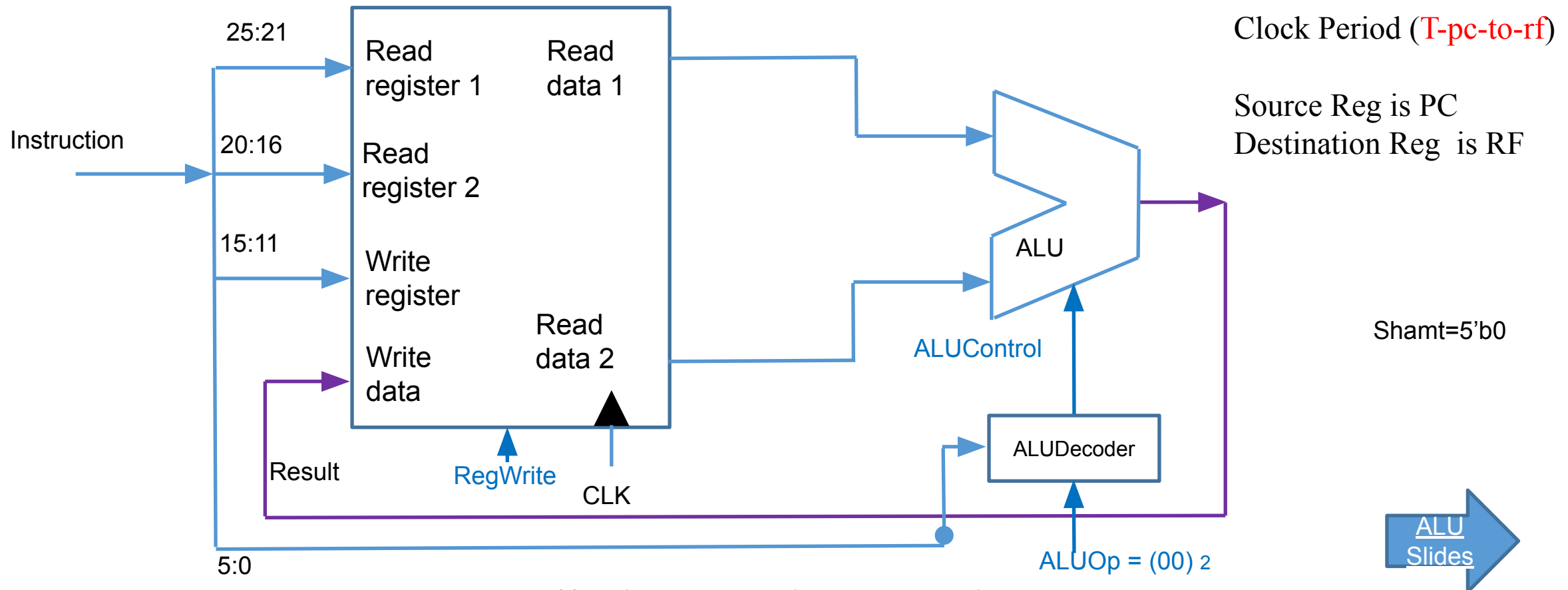
Source Reg is PC
Destination Reg is PC

$IMM[PC]; PC = PC + 4$

Analysis of data-path for R-type instruction

- ADD \$S1, \$S2, \$S3 // \$S1 $\square$ \$S2 + \$S3

op	rs	rt	rd	shamt	funct
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



Analysis of data-path for I-type instruction

- LW \$S1, offset[\$S2] //\$S1 $\square$ DM[offset + \$S2]

Is offset a physical address?

No. It is a relative address (here, relative with respect to Reg.)

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

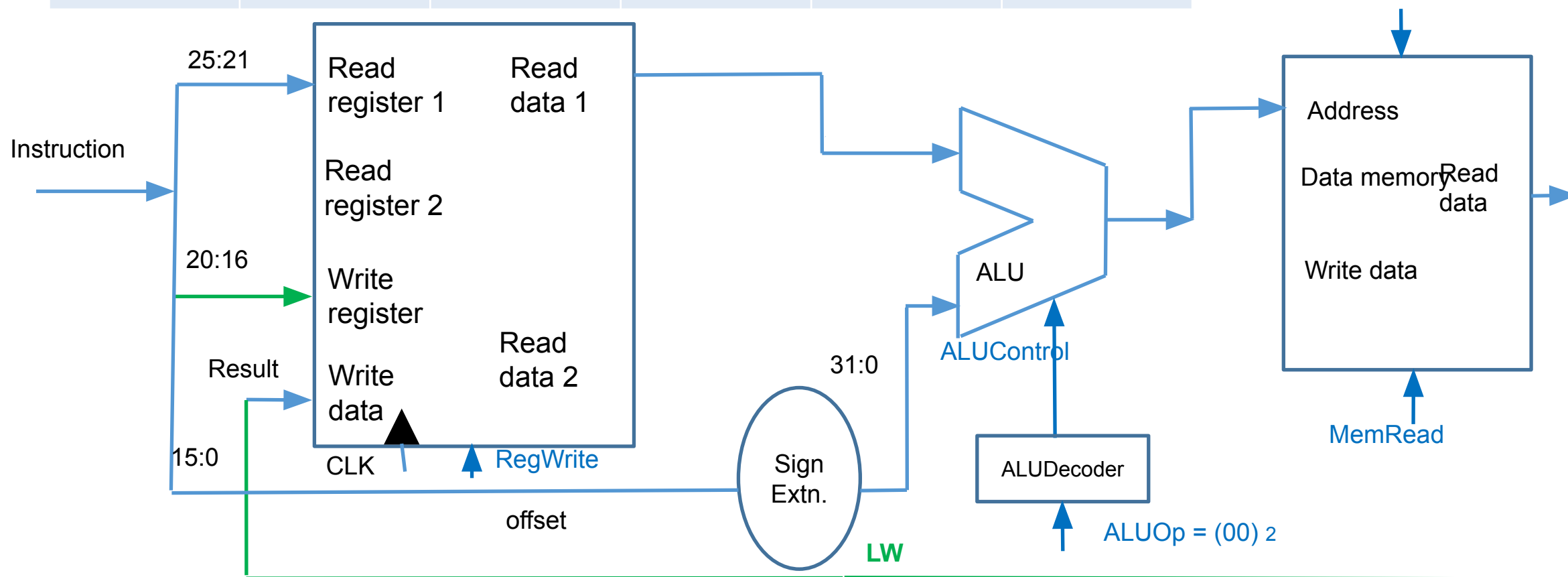
Analysis of data-path for I-type instruction

- LW \$S1, offset[\$S2] // \$S1 ← DM[offset + \$S2]

Is offset a physical address?

No. It is a relative address (here, relative with respect to Reg.)

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



How does one measure the clock period?

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

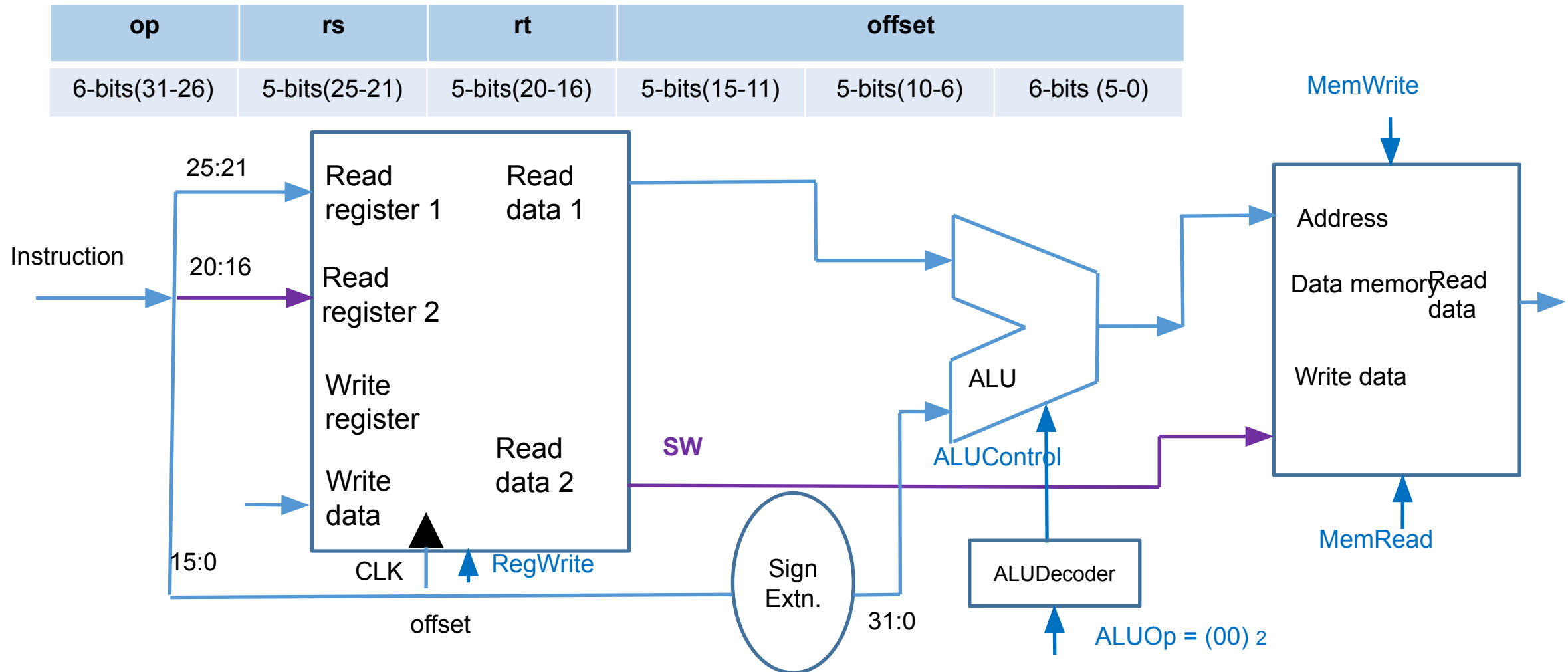
Analysis of data-path for I-type instruction

- SW \$S1, offset[\$S2] //DM[offset + \$S2] □ \$S1

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

Analysis of data-path for I-type instruction

- SW \$S1, offset(\$S2) //DM[offset + \$S2] ← \$S1



How does one measure the clock period?

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Analysis of data-path for I-type instruction

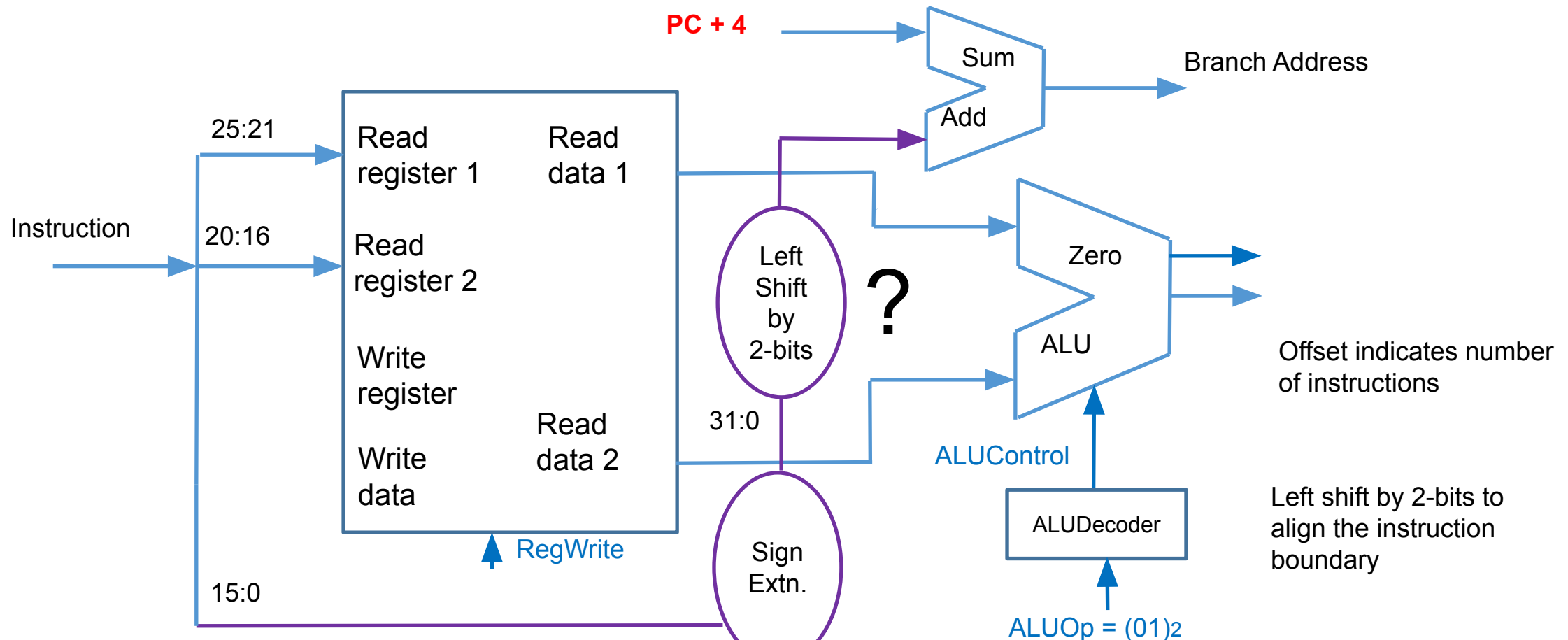
- BEQ \$S1, \$S2, offset //Jump to the offset no. of instr., when \$S1 = \$S2
- BNE \$S1, \$S2, offset //Jump to the offset no. of instr., when \$S1 != \$S2

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

Analysis of data-path for l-type instruction

- `BEQ $S1, $S2, offset` //Jump to the offset no. of instr., when `$S1 = $S2`
- `BNE $S1, $S2, offset` //Jump to the offset no. of instr., when `$S1 != $S2`

op	rs	rt	offset		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



How does one measure the clock period?

(c) Kanchan Manna; BITS Pilani, Goa Campus, India.

Analysis of data-path I-type instruction

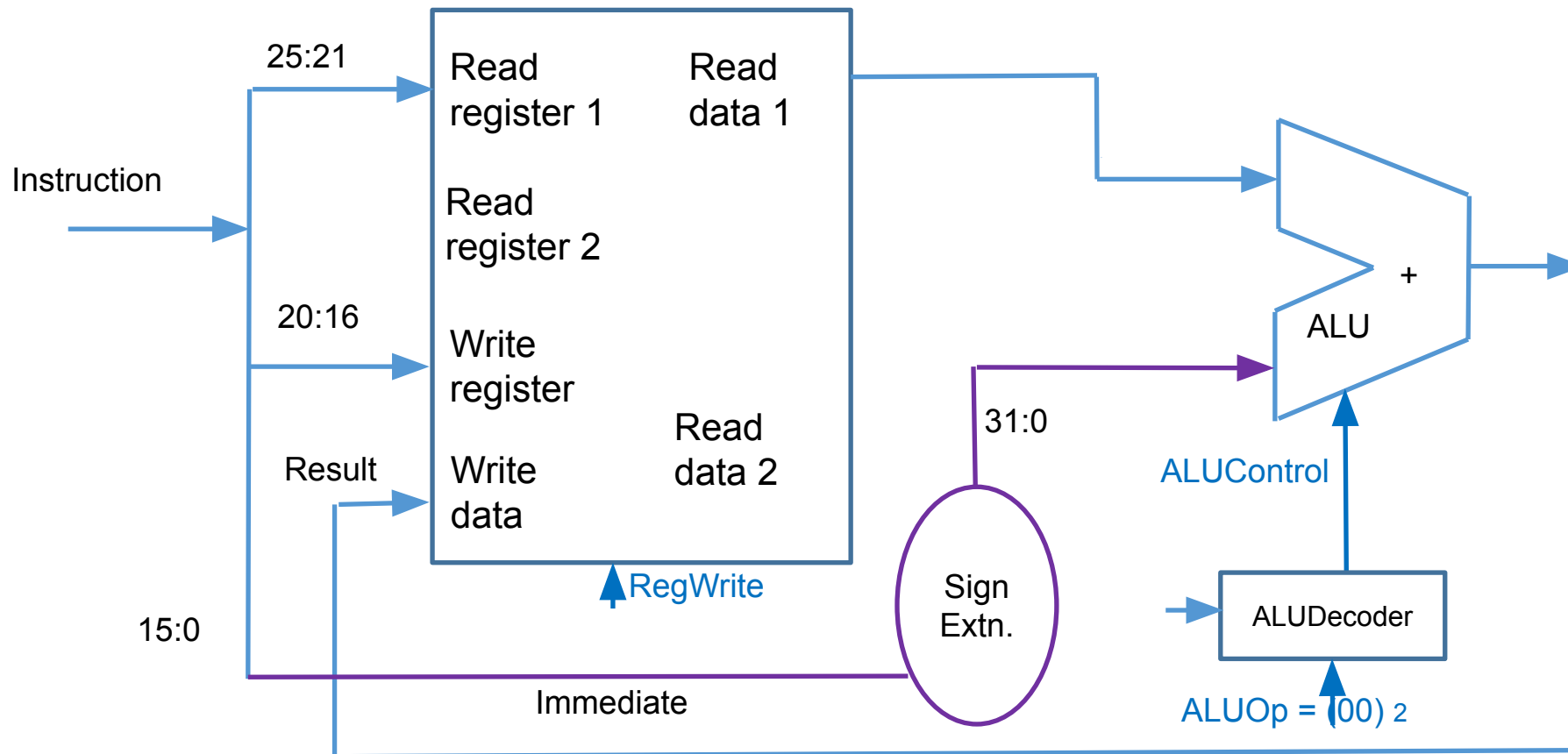
- ADDI \$S1, \$S2, -12 // \$S1 $\leftarrow$ \$S2 + (-12)

op	rs	rd	Immediate		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

Analysis of data-path I-type instruction

- `ADDI $S1, $S2, -12` // $\$S1 \leftarrow \$S2 + (-12)$

op	rs	rd	Immediate		
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



How does one measure the clock period?

(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Analysis of data-path j-type instruction

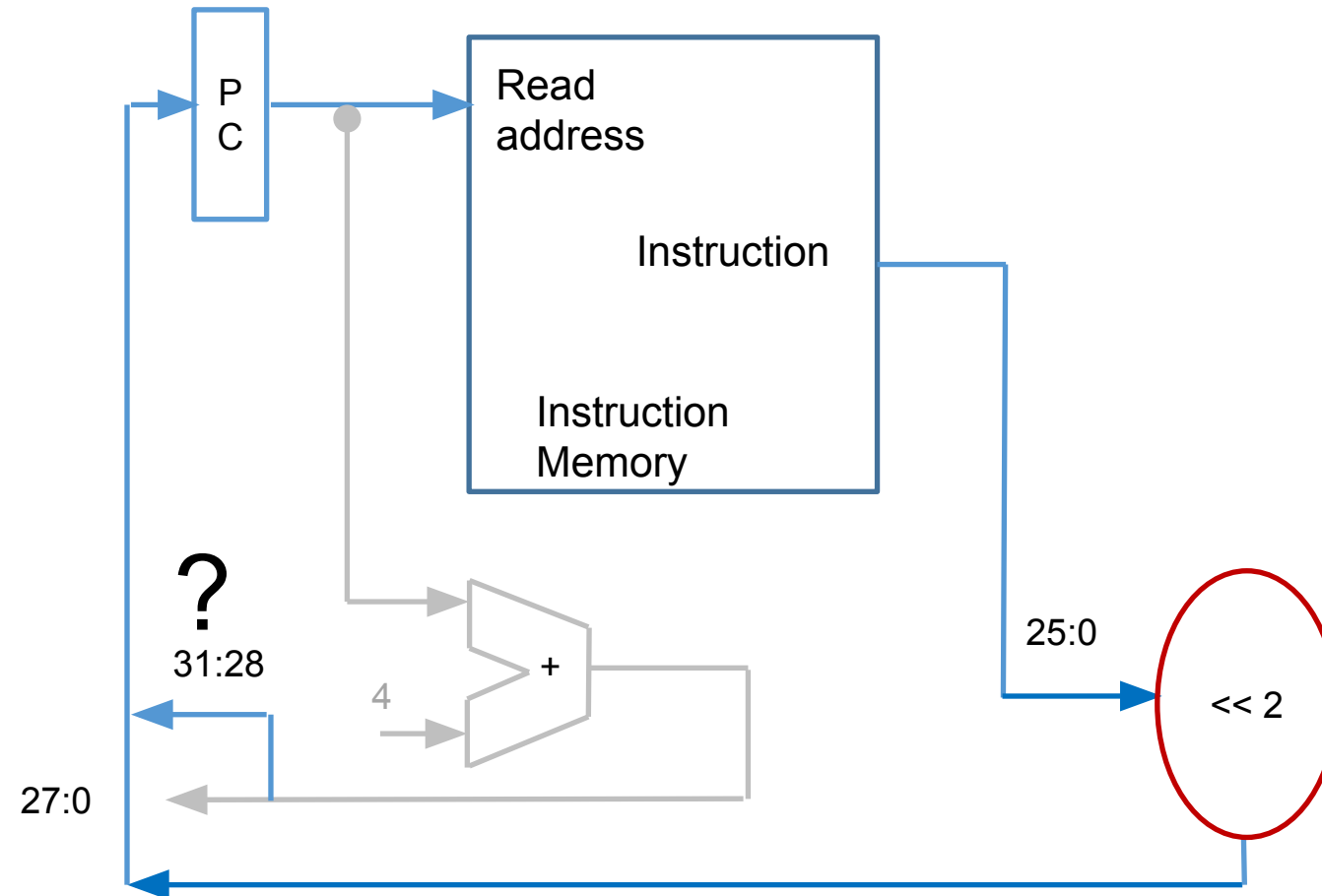
- J addr_s // PC $\square$ PC[31:28]addr_s[27:0]

op	address				
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)

Analysis of data-path j-type instruction

- J addr // PC $\square$ PC[31:28]addr[27:0]

op	address				
6-bits(31-26)	5-bits(25-21)	5-bits(20-16)	5-bits(15-11)	5-bits(10-6)	6-bits (5-0)



How does one measure the clock period?

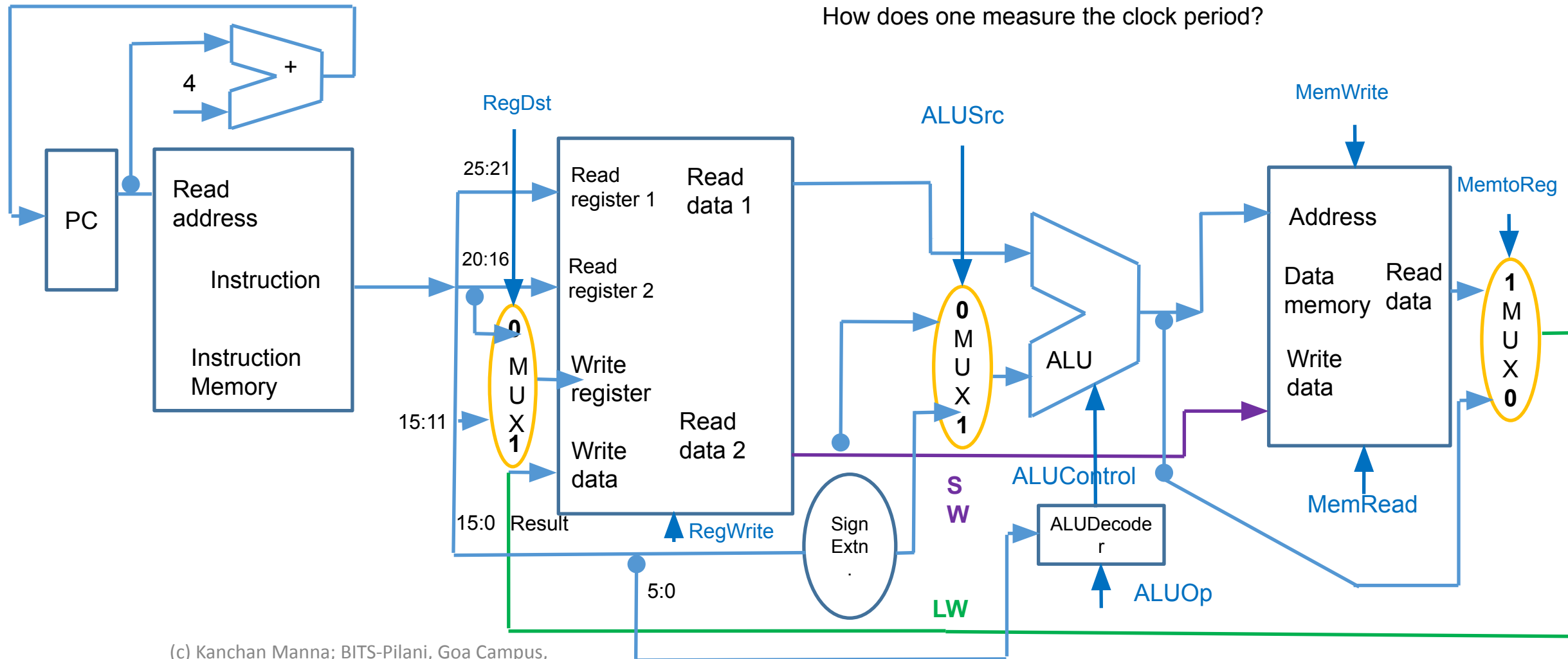
(c) Kanchan Manna; BITS-Pilani, Goa Campus, India.

Building Microprocessor

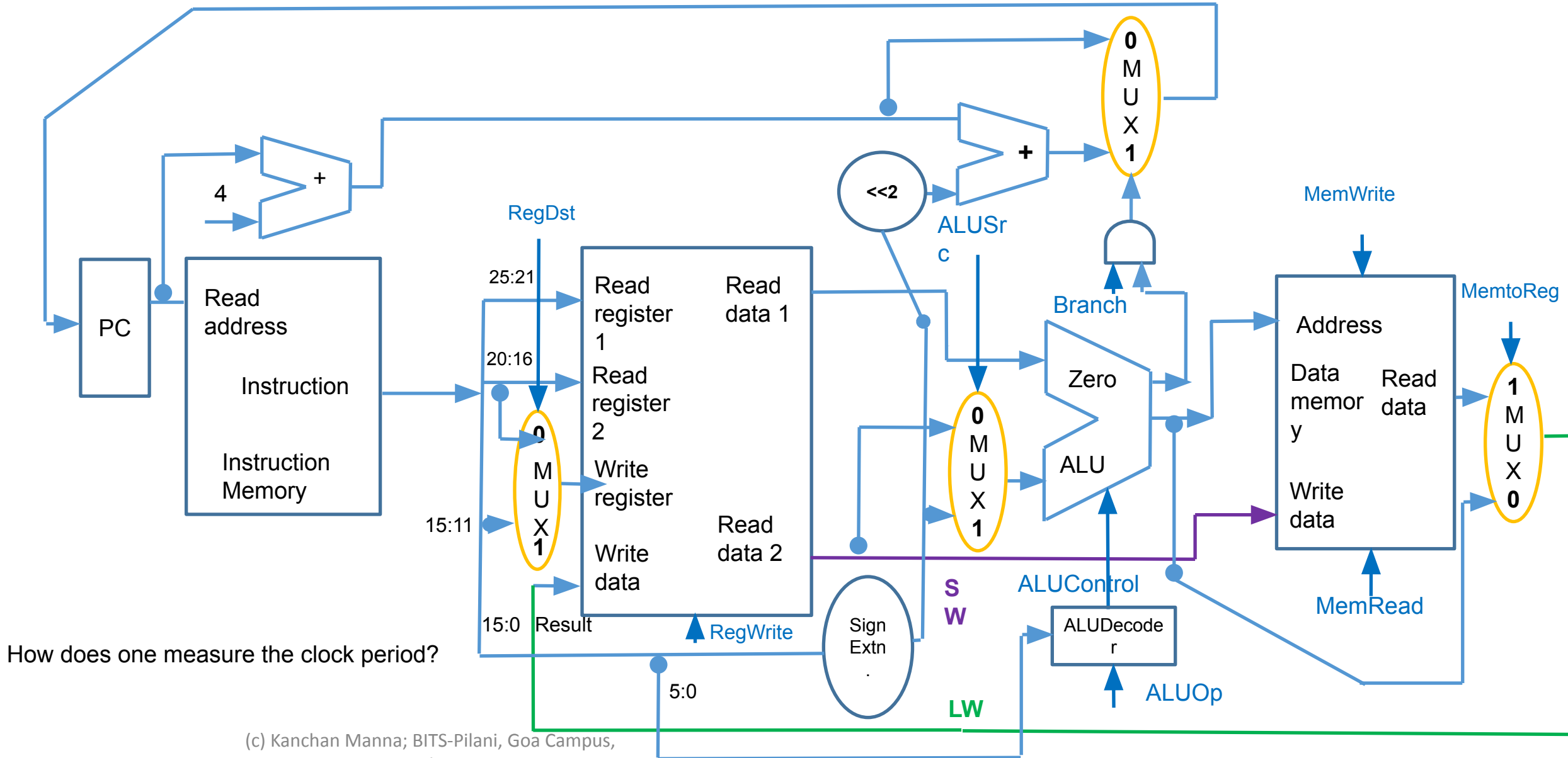
- Designed the individual datapath for
 - Instruction Fetch
 - R-type instructions
 - I-type instructions
 - J-type instructions
- How does one build the microprocessor with these instructions?
 - Merge the datapath for all the instructions type, including the fetch, **How?**
 - Using MUX

Combined Fetch cycle, R, and I-type data-path

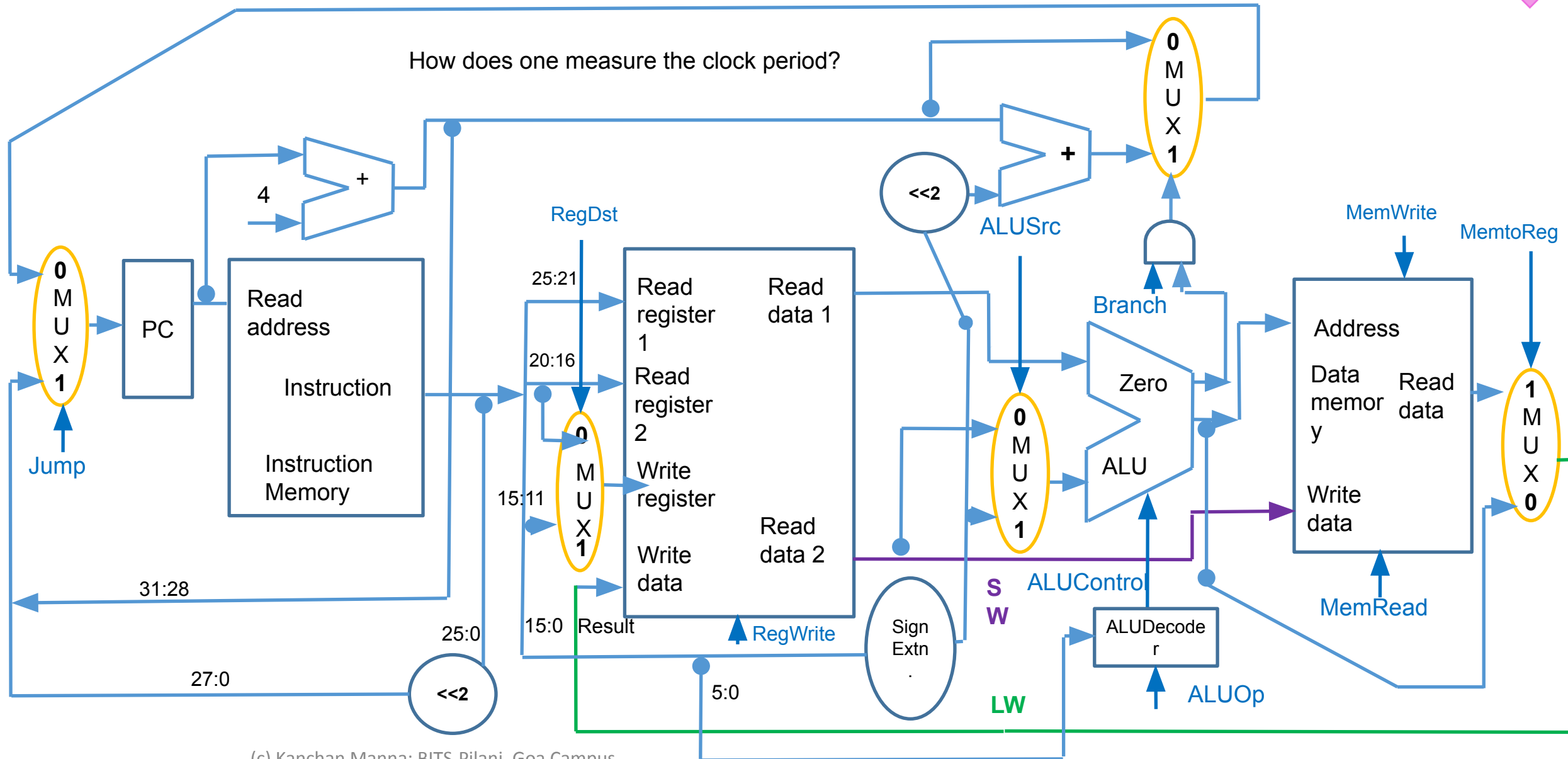
- For ALU and write register, source of data, for the input, is more than one
- Insert MUX before such input signal and control the inputs through MUX-select line



Combined Fetch cycle, R, I and Branch instr data-path



Combined Fetch cycle, R, I and J-type data-path



Identify the control signals

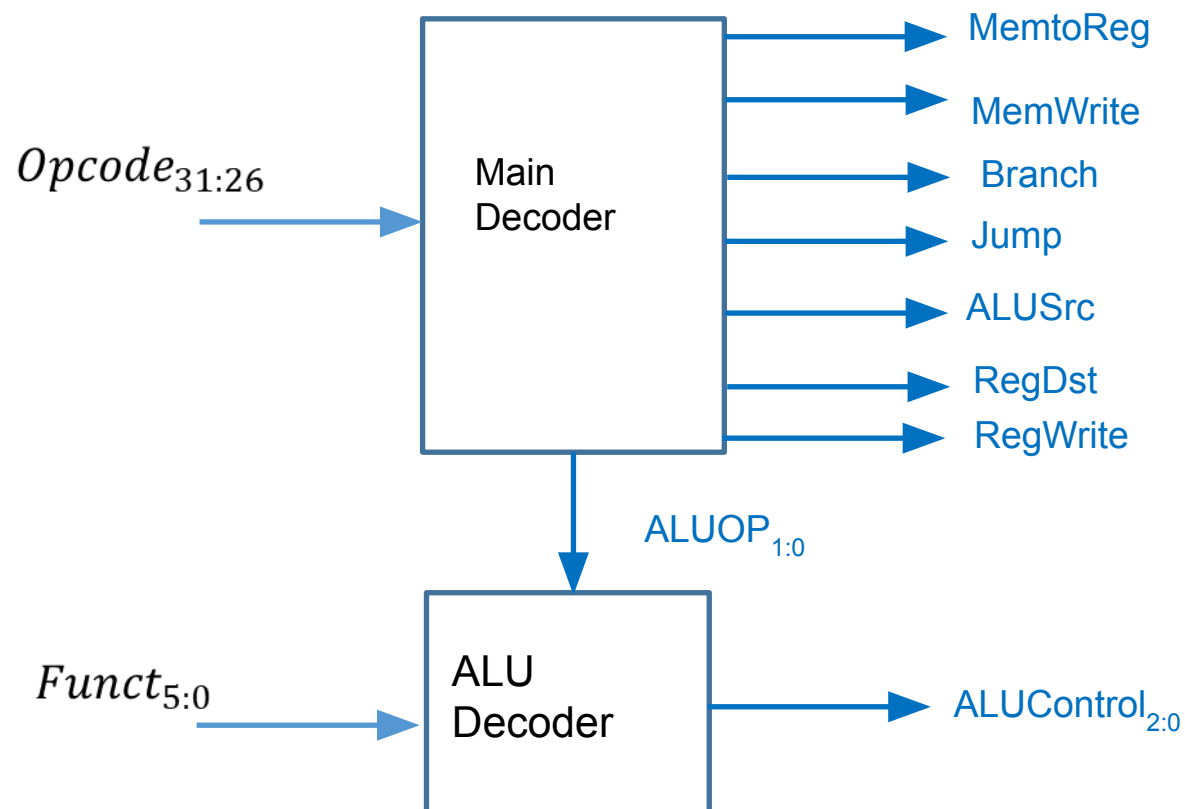
- Jump
- RegDst
- RegWrite
- ALUSrc
- Branch
- ALUOp
- MemRead
- MemWrite
- MemtoReg

ALUOp	Meaning
00	add
01	subtract
10	Look at <i>funct</i> field
11	n/a

How can we design a Controller?

Control Unit

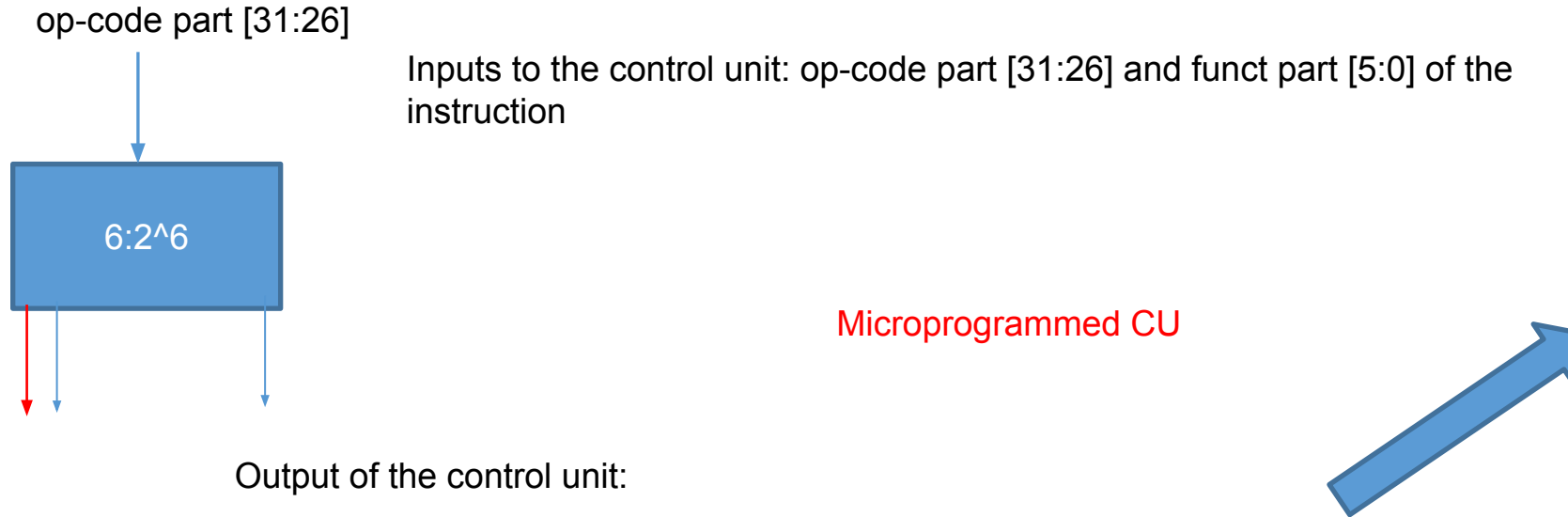
Inputs



Outputs

ALUOp	Meaning
00	add
01	subtract
10	Look at <i>funct</i> field
11	n/a

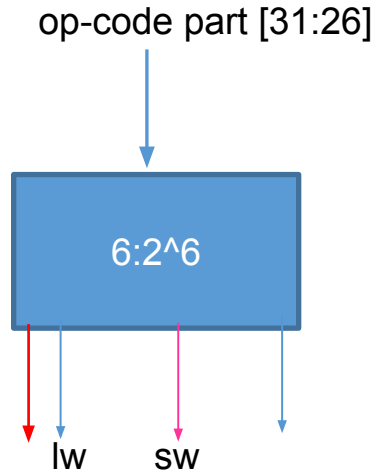
Generation of Controls: **Main decoder** truth table



ALUOp	Meaning
00	add
01	subtract
10	Look at <i>funct</i> field
11	n/a

Instr. (Input)	Jump	RegDst	RegWrite	ALUSrc	Branch	ALUOp1	ALUOp0	MemRead	MemWrite	MemtoReg
R-type	0	1	1	0	0	1	0	0	0	0
lw	0	0	1	1	0	0	0	1	0	1
sw	0	x	0	1	0	0	0	0	1	x
addi	0	0	1	1	0	0	0	0	0	0
B-type	0	x	0	0	1	0	1	0	0	x
J-type	1	x	0	x	x	x	x	0	0	x

Generation of Controls: **Main decoder** truth table



Inputs to the control unit: op-code part [31:26] and funct part [5:0] of the instruction

Consider **ALUSrc**

What could be the C-expression?

If ((lw) || (sw) || (addi))

ALUSrc = 1;

Else

ALUSrc = 0;

Output of the control unit:

Instr. (Input)	Jump	RegDst	RegWrite	ALUSrc
R-type	0	1	1	0
lw	0	0	1	1
sw	0	x	0	1
addi	0	0	1	1
B-type	0	x	0	0
J-type	1	x	0	x

How can we write it using Logical expression?

$ALUSrc = (lw) + (sw) + (addi)$

Hardwired CU

ALU Operations

ALU control lines (3:0)	ALU Functions
0000	AND
0001	OR
0010	Add
0110	Subtract
0111	set on less than

Generation of Control: **ALU decoder** truth table

ALUOp	ALU control lines (2:0)	Funct
00	010 (add)	X
01	110 (subtract)	X
1X	010 (add)	100000 (add)
1X	110 (subtract)	100010 (sub)
1X	000 (and)	100100 (and)
1X	001 (or)	100101 (or)
1X	111 (set less than)	101010 (slt)

Generation of Controls

Inst. opcode	ALUOp	Instr. operation	Funct field	Desired ALU action	ALUControl
100010 (LW)	00	load word	xxxxxx	add	0010
100011 (SW)	00	store word	xxxxxx	add	0010
000100 (BEQ)	01	branch equal	xxxxxx	subtract	0110
000000 (R-type)	10	add	100000	add	0010
R-type	10	Subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111
001000 (addi)	00	Immediate	xxxxxx	add	xxxx
000010 (j)	xx	jump	xxxxxx	jump	xxxx

Performance analysis implementation

- Execution time of a program is a metric
- $Execution\ Time = (\#instructions) \left(\frac{Cycles}{instruction} \right) \left(\frac{Seconds}{cycle} \right)$
- #instructions or length of a program depends on ISA
- Complicated Vs. Simple ISA
- The number of cycle per instruction (on an average) is called *CPI*
- Throughput = $1/CPI$
- Assumption: an ideal memory model
- The number of seconds per cycle is the clock period (?)

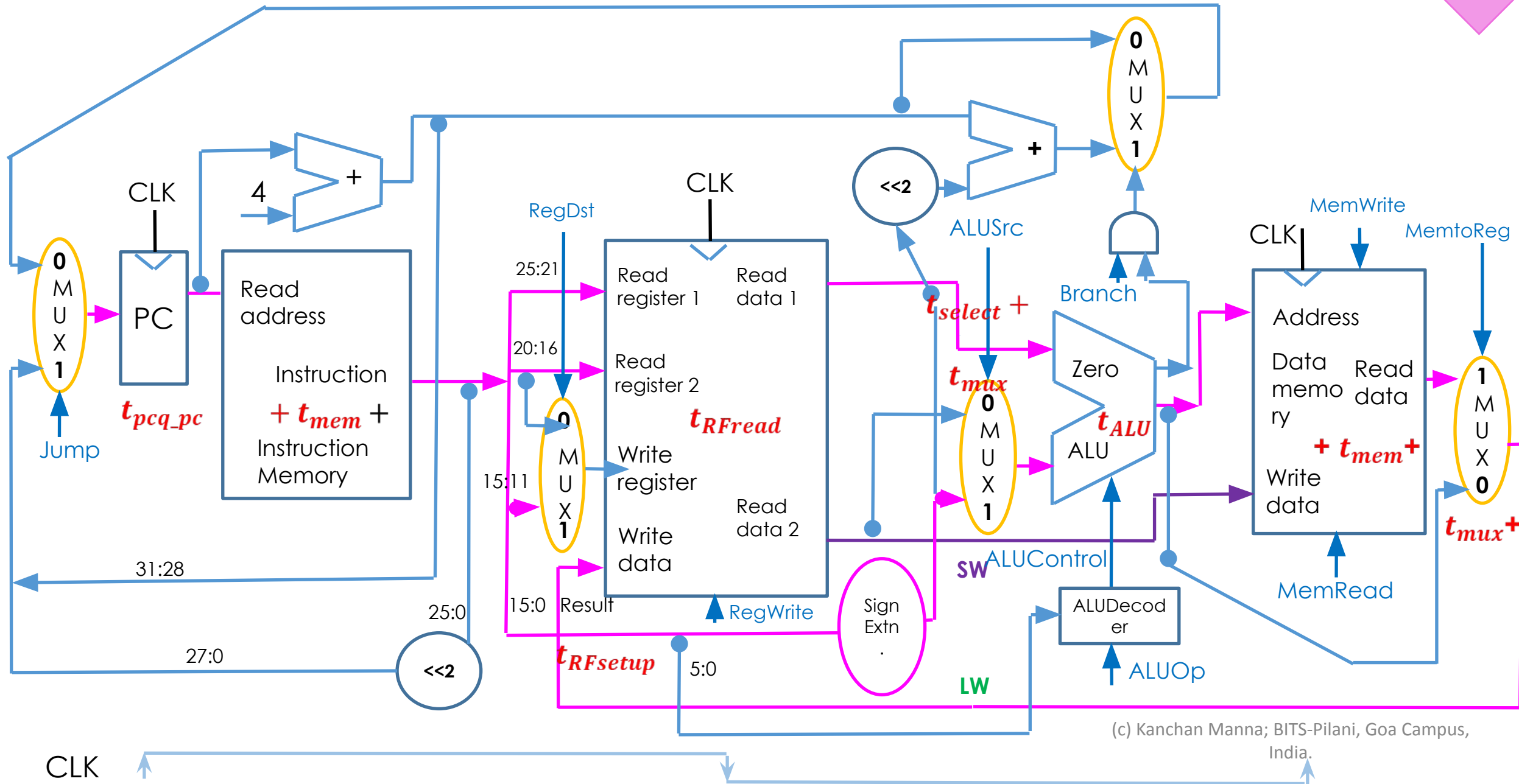
Performance analysis of implementation

Instr.	Resource Usage										
	PC	IMM	ADD_PC	RF	EXTN	ALU	<<2	ADD_B	DMM	RF (Write)	Total Resource Usage
ADD	✓	✓	✓	✓	×	✓	×	×	×	✓	6
BNE	✓	✓	✓	✓	✓	✓	✓	✓	×	×	8
J	✓	✓	✓	×	×	×	×	×	×	×	3
SW	✓	✓	✓	✓	✓	✓	×	×	✓	×	7
LW	✓	✓	✓	✓	✓	✓	×	×	✓	✓	8
ADDI	✓	✓	✓	✓	✓	✓	×	×	×	✓	7

What could be maximum time to update the PC? Because updated PC will bring the next instruction in the datapath

Which of the instruction using maximum resource? Next consider the time taken by each resource to compute the functionality

Consider the **pink lines** for **LW's path**



Performance analysis of Single-cycle implementation

- CPI = 1, for single-cycle implementation
- lw-instruction decides Critical path (T_c)
 - $T_c = t_{pcq_PC} + t_{mem} + \max\{t_{RFrear}, t_{select} + t_{mux}\} + t_{ALU} + t_{mem} + t_{mux} + t_{RFwrite}$
- Register read takes longer time than mux selection
- $T_c = t_{pcq_PC} + 2t_{mem} + t_{RFrear} + t_{ALU} + t_{mux} + t_{RFwrite}$

Single-cycle implementation

- The previous design is called single-cycle implementation
- The instruction memory, register file and data memory are all **read combinational**
 - What does it mean?
 - The new instruction appears to output of instruction memory after some propagation delay, if the **address changes**
- Operations are done on rising edge of the clock
- The single-cycle microarchitecture executes an entire instruction in one clock cycle
- Simple control unit (why?)
 - No next state is associated with it
 - Every operation is done in a clock cycle

Performance analysis of Single-cycle implementation

- Need some quantity (or metric) for comparison of two design
- How does one measure the effectiveness of new design?

Performance analysis of Single-cycle implementation

- How to calculate Delay
- Delay: time between applying the Input and producing the output, another way to say time between two inputs, i.e., when we can update the PC
 - Input is reading an instruction from Memory
 - Output is producing the result by the read instruction
 - Next input is available when we update the PC by $(PC + 4)$ or Branch address

Performance analysis of Single-cycle implementation

- XYZ-organization is going to build the single-cycle MIPS processor in a 65-nm CMOS manufacturing process. The organization has determined that the logic elements have the delays given in Table. Help the organization compute the execution time for a program with 100 billion instructions.

Parameter	Delay (ps)
	30
	250
	20
	200
	25
	20

Performance analysis of Single-Cycle implementation

- XYZ-organization is contemplating building the single-cycle MIPS processor in a 65-nm CMOS manufacturing process. The organization has determined that the logic elements have the delays given in Table. Help the organization compute the execution time for a program with 100 billion instructions.
- $$T_c = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFSetup}$$

$$= 30 + 2(250) + 150 + 200 + 25 + 20 = \mathbf{925\ ps}$$
- The total execution time =

$$(100 * 10^9 \text{ instrs.}) * (1 \text{ cycle/instrs.}) * (925 * 10^{-12} \text{ s/cycle})$$

$$= 92.5 \text{ seconds}$$

Parameter	Delay (ps)
	30
	250
	150
	200
	25
	20

Other Benchmark: Standard Performance Evaluation Corporation (SPEC)
 Princeton Application Repository for Shared-Memory Computers (PARSEC)
 Stanford Parallel Applications for Shared-Memory (SPLASH-2) programs.

A few more MIPS instructions

MIPS ISA & Load Byte (lb)

lui	rt, imm	Load Upper Imm.	$rt = \text{imm} \ll 16$
lb	rt, imm(rs)	Load Byte	$rt = \text{SignExt}(M_1[rs + \text{imm}_{\pm}])$
lbu	rt, imm(rs)	Load Byte Unsigned	$rt = M_1[rs + \text{imm}_{\pm}] \& 0xFF$
lh	rt, imm(rs)	Load Half	$rt = \text{SignExt}(M_2[rs + \text{imm}_{\pm}])$
lhu	rt, imm(rs)	Load Half Unsigned	$rt = M_2[rs + \text{imm}_{\pm}] \& 0xFFFF$
lw	rt, imm(rs)	Load Word	$rt = M_4[rs + \text{imm}_{\pm}]$
sb	rt, imm(rs)	Store Byte	$M_1[rs + \text{imm}_{\pm}] = rt$
sh	rt, imm(rs)	Store Half	$M_2[rs + \text{imm}_{\pm}] = rt$
sw	rt, imm(rs)	Store Word	$M_4[rs + \text{imm}_{\pm}] = rt$

MIPS ISA can be found:

<https://www.scribd.com/document/358717972/mips-ref-pdf>

MIPS ISA & Load Byte (lb)

lui	rt, imm	Load Upper Imm.	$rt = \text{imm} \ll 16$
lb	rt, imm(rs)	Load Byte	$rt = \text{SignExt}(M_1[rs + \text{imm}_{\pm}])$
lbu	rt, imm(rs)	Load Byte Unsigned	$rt = M_1[rs + \text{imm}_{\pm}] \& 0xFF$
lh	rt, imm(rs)	Load Half	$rt = \text{SignExt}(M_2[rs + \text{imm}_{\pm}])$
lhu	rt, imm(rs)	Load Half Unsigned	$rt = M_2[rs + \text{imm}_{\pm}] \& 0xFFFF$
lw	rt, imm(rs)	Load Word	$rt = M_4[rs + \text{imm}_{\pm}]$
sb	rt, imm(rs)	Store Byte	$M_1[rs + \text{imm}_{\pm}] = rt$
sh	rt, imm(rs)	Store Half	$M_2[rs + \text{imm}_{\pm}] = rt$
sw	rt, imm(rs)	Store Word	$M_4[rs + \text{imm}_{\pm}] = rt$

MIPS ISA can be found:

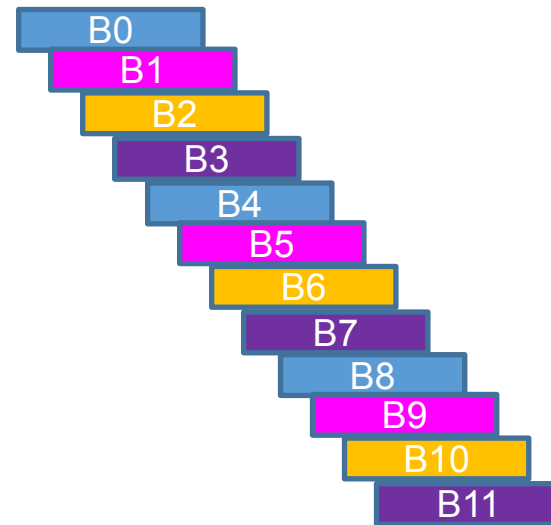
<https://www.scribd.com/document/358717972/mips-ref-pdf>

[MIPS Reference Card \(yumpu.com\)](https://www.yumpu.com/document/view/358717972/mips-ref-pdf)

MIPS ISA & Load Byte (lb)

Name	Instr	Type	Operation
Load Byte	lb rt, imm(rs)	I	$RF[rt] = \text{Signextn}(M[RF[rs] + \text{signextn}(Imm)](7:0))$
Load Byte Unsigned	lbu rt, imm(rs)	I	$RF[rt] = \{24'b0, M[RF[rs] + \text{signextn}(Imm)](7:0)\}$
Load Half-word	lh rt, imm(rs)	I	$RF[rt] = \text{Signextn}(M[RF[rs] + \text{signextn}(Imm)](15:0))$
Load Half-word Unsigned	lhu rt, imm(rs)	I	$RF[rt] = \{16'b0, M[RF[rs] + \text{signextn}(Imm)](15:0)\}$

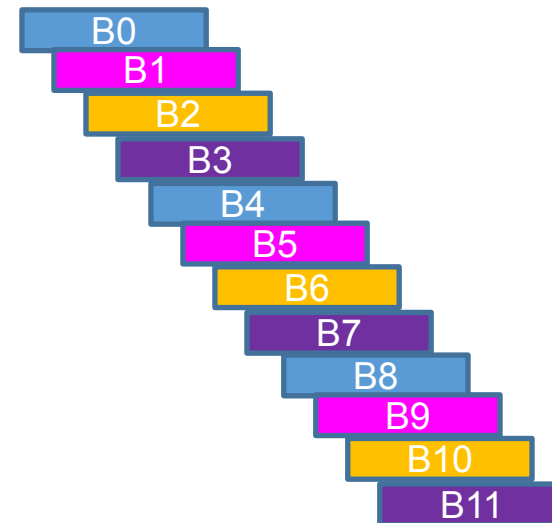
MIPS ISA & Load Byte (lb)



MIPS ISA & Store Byte (lb)

81

Name	instrn	Type	Operation
Store Byte	sb rt, imm(rs)	I	$M[RF[rs] + \text{signextn}(\text{Imm})](7:0) = RF[rt](7:0)$
Store Half-word	sh rt, imm(rs)	I	$M[RF[rs] + \text{signextn}(\text{Imm})](15:0) = RF[rt](15:0)$



Datapath and control?

Shift instructions in MIPS

Assembly Code

sll \$t0, \$s1, 4

srl \$s2, \$s1, 4

sra \$s3, \$s1, 4

Field Values

op	rs	rt	rd	shamt	funct
0	0	17	8	4	0
0	0	17	18	4	2
0	0	17	19	4	3
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Machine Code

op	rs	rt	rd	shamt	funct	
000000	00000	10001	01000	00100	000000	(0x00114100)
000000	00000	10001	10010	00100	000010	(0x00119102)
000000	00000	10001	10011	00100	000011	(0x00119903)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Shift instructions in MIPS

Assembly Code

Field Values

	op	rs	rt	rd	shamt	funct
sllv \$s3, \$s1, \$s2	0	18	17	19	0	4
srlv \$s4, \$s1, \$s2	0	18	17	20	0	6
sra v \$s5, \$s1, \$s2	0	18	17	21	0	7
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

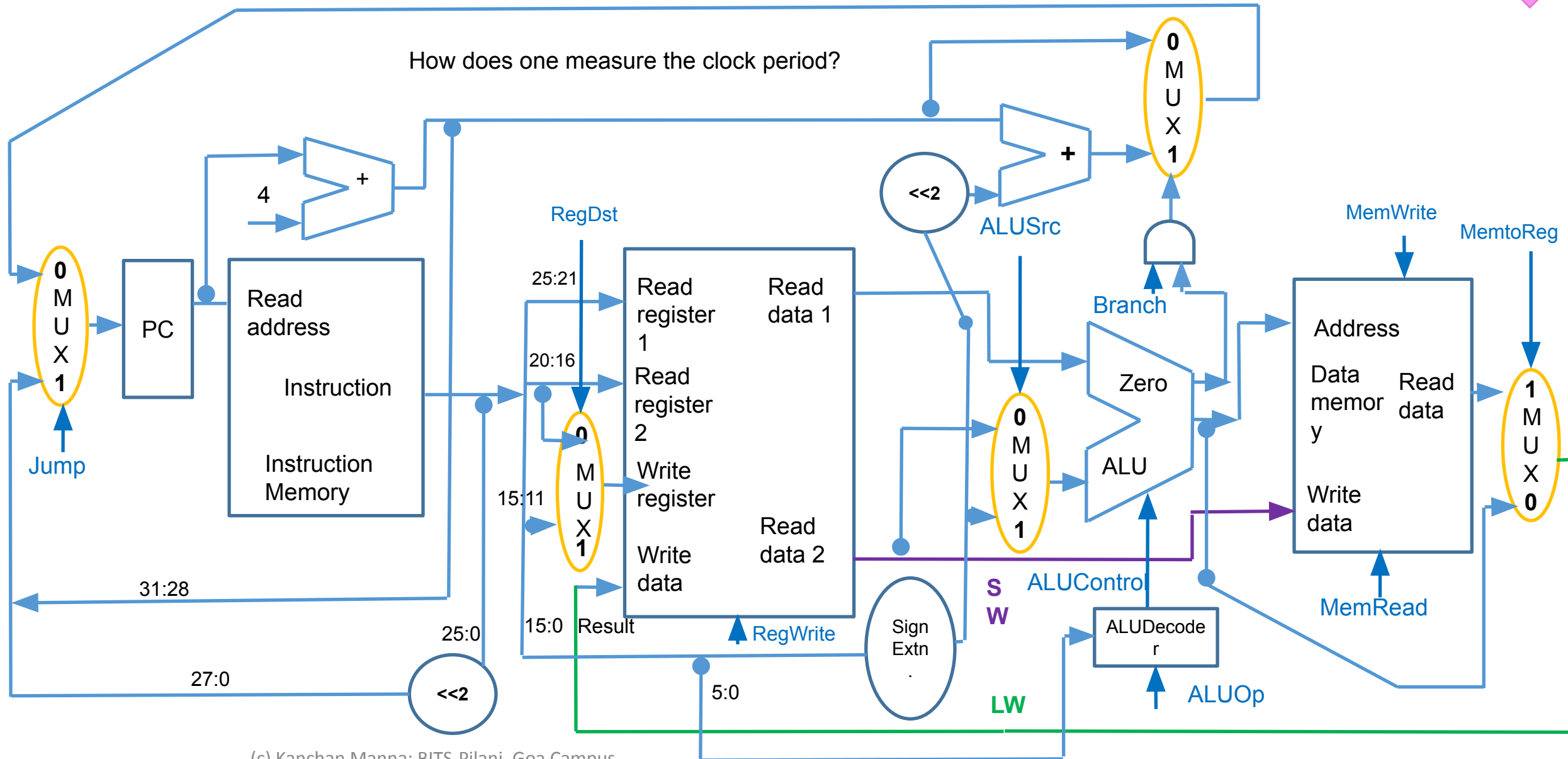
Machine Code

op	rs	rt	rd	shamt	funct	
000000	10010	10001	10011	00000	000100	(0x02519804)
000000	10010	10001	10100	00000	000110	(0x0251A006)
000000	10010	10001	10101	00000	000111	(0x0251A807)
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	

Shift instructions in MIPS

		Source Values							
\$s1		1111	0011	0000	0000	0000	0010	1010	1000
shamt									00100
Assembly Code		Result							
sll \$t0, \$s1, 4	\$t0	0011	0000	0000	0000	0010	1010	1000	0000
srl \$s2, \$s1, 4	\$s2	0000	1111	0011	0000	0000	0000	0010	1010
sra \$s3, \$s1, 4	\$s3	1111	1111	0011	0000	0000	0000	0010	1010

Combined Fetch cycle, R, I and J-type data-path



Find the minimum and maximum number from a set of numbers

MIPS: Microprocessor without Interlocked Pipelined Stages

```
.data
array:    .word  1, 2, -8, 0, 23, 11, -10
array_size: .word  10
array_min: .ascii "\nMin: "
array_max: .ascii "\nMax: "
minE:      .word  999
maxE:      .word -999

.text
main:
    la $a0, array
    lw $a1, array_size
    lw $t2, maxE # max
    lw $t3, minE # min
```

Find the minimum and maximum number from a set of numbers

loop_array:

beq \$a1, \$zero, print_and_exit

lw \$t0, (\$a0)

bge \$t0, \$t3, not_min # if (current_element >= current_min) {don't modify min}

move \$t3, \$t0

not_min:

ble \$t0, \$t2, not_max # if (current_element <= current_max) {don't modify max}

move \$t2, \$t0

not_max:

addi \$a1, \$a1, -1

addi \$a0, \$a0, 4

j loop_array

Find the minimum and maximum number from a set of numbers

```
print_and_exit:
```

```
    # print minimum
```

```
    li $v0, 4 #for string
```

```
    la $a0, array_max
```

```
    syscall
```

```
    li $v0, 1 #for number
```

```
    move $a0, $t2
```

```
    syscall
```

```
# print maximum
```

```
    li $v0, 4
```

```
    la $a0, array_min
```

```
    syscall
```

```
    li $v0, 1
```

```
    move $a0, $t3
```

```
    syscall
```

```
# exit
```

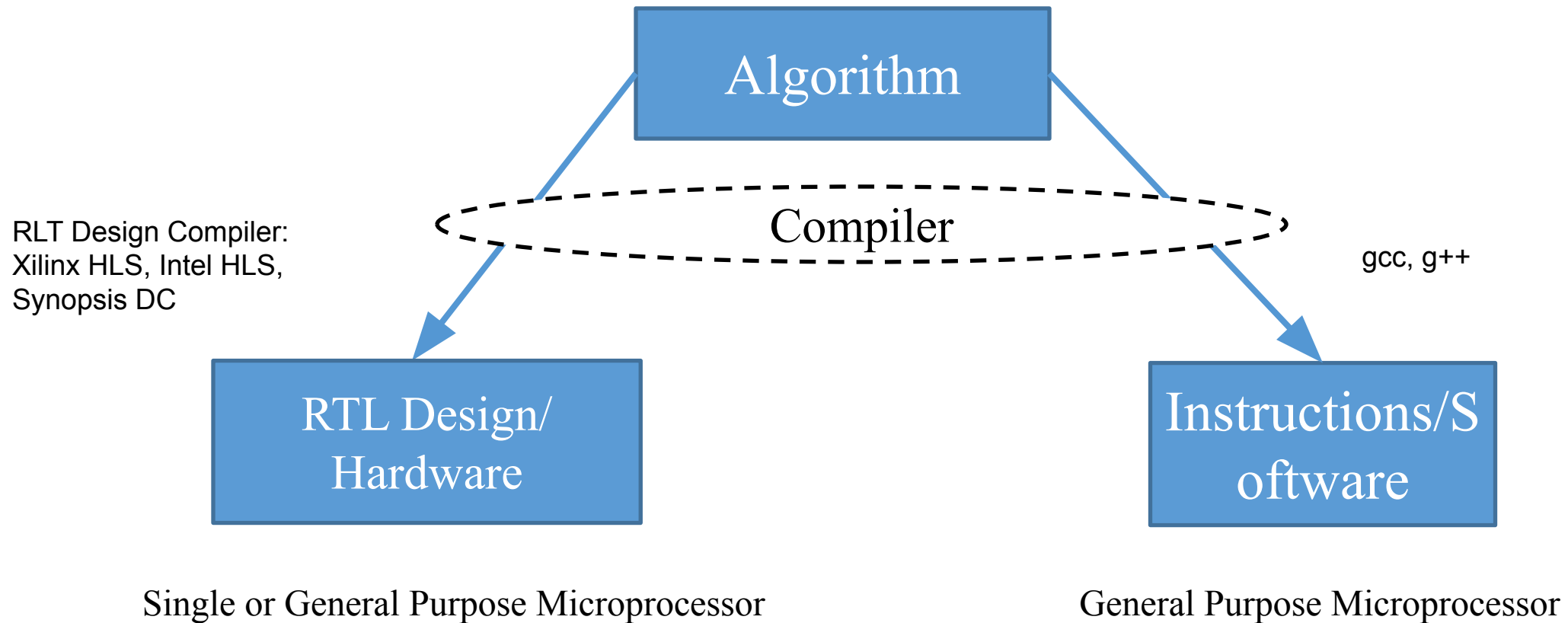
```
    li $v0, 10
```

```
    syscall
```

Find the minimum and maximum number from a set of numbers

```
int main()
{
    int arr[10] = {1, 2, -8, 0, 23, 11, -10};
    int N = 10, i;
    int minE = 9999, maxE = -9999;
    // Traverse the given array
    for (i = 0; i < N; i++) {
        // If current element is smaller than minE then update it
        if (arr[i] < minE) {
            minE = arr[i];
        }
        // If current element is greater than maxE then update it
        if (arr[i] > maxE) {
            maxE = arr[i];
        }
    }
    printf("The minimum element is %d", minE); printf("\n");
    printf("The maximum element is %d", maxE);
    return 0;
}
```

Algorithm and its Possible Architectures



How does the General-purpose micro-processor solve the problem?

Problem

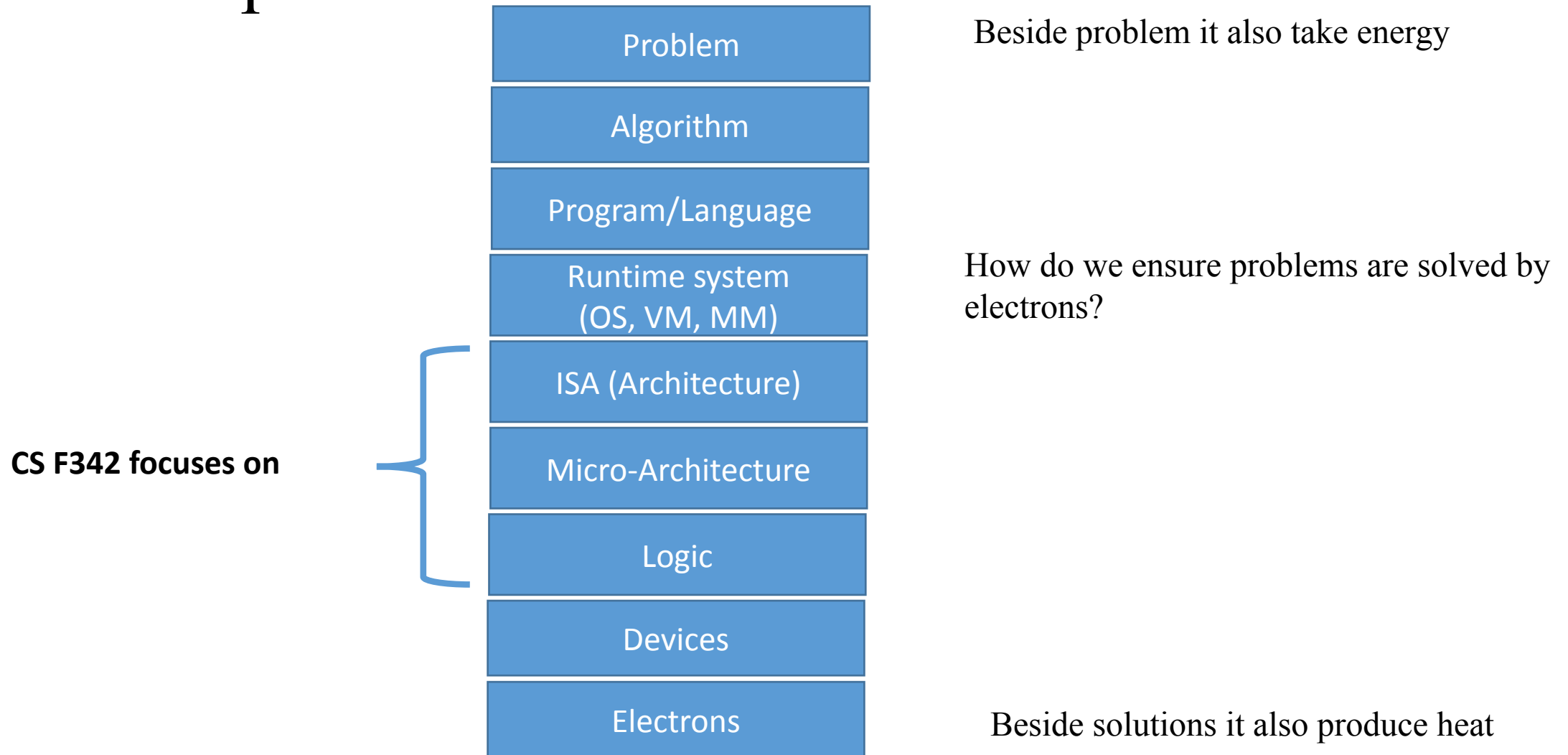
Beside problem it also take energy

How do we ensure problems are solved by electrons?

Electrons

Beside solutions it also produce heat

How does the General-purpose micro-processor solve the problem?



What is the basic building block of a program?

- A C-program: min_max.c
- Perform: gcc -S min_max.c
- Assembly code: min_max.s
- The basic building block of a program is **instructions**
- What is the significance of instruction order?
- From a program's (compiler writer's) point of view the computer is the instruction set
- Instruction Set Architecture can be MIPS (32, 64), RISC-V (32, 64), 8085, x86 (refers to a 16/32-bit CPU of type 8086), x64 (refers to a 64-bit CPU of type 8086)
- All program use same set of instructions

What is the meaning of Computer Architecture?

- Architecture: Computational (dedicated or general-purpose) structure with respect to user (programmer, etc)
 - Use of minimal resources
 - Easily scalable
- For an example, a set of instruction can be the meaning of comp. arch. to a programmer
- A dedicated processor
- Algorithm has its physical structure

Two Very Important Ideas

- **Idea 1:** All computers (the biggest and the smallest, the fastest and the slowest, the most expensive and the cheapest) are capable of computing exactly the same things if they are given enough time and enough memory.
- **Idea 2:** We describe our problems in English or some other language spoken by people. Yet the problems are solved by electrons running around inside the computer. It is necessary to transform our problem from the language of humans to the voltages that influence the flow of electrons.

History of computation

- Homework
- Go through the material on Gdrive, shared with you.
- <https://drive.google.com/drive/folders/1JkTqBCwtP8o6-7YzjX8JfxbabKT8puzW>
- Go through the order mentioned in the xlsx file
- Will ask the question in the next class

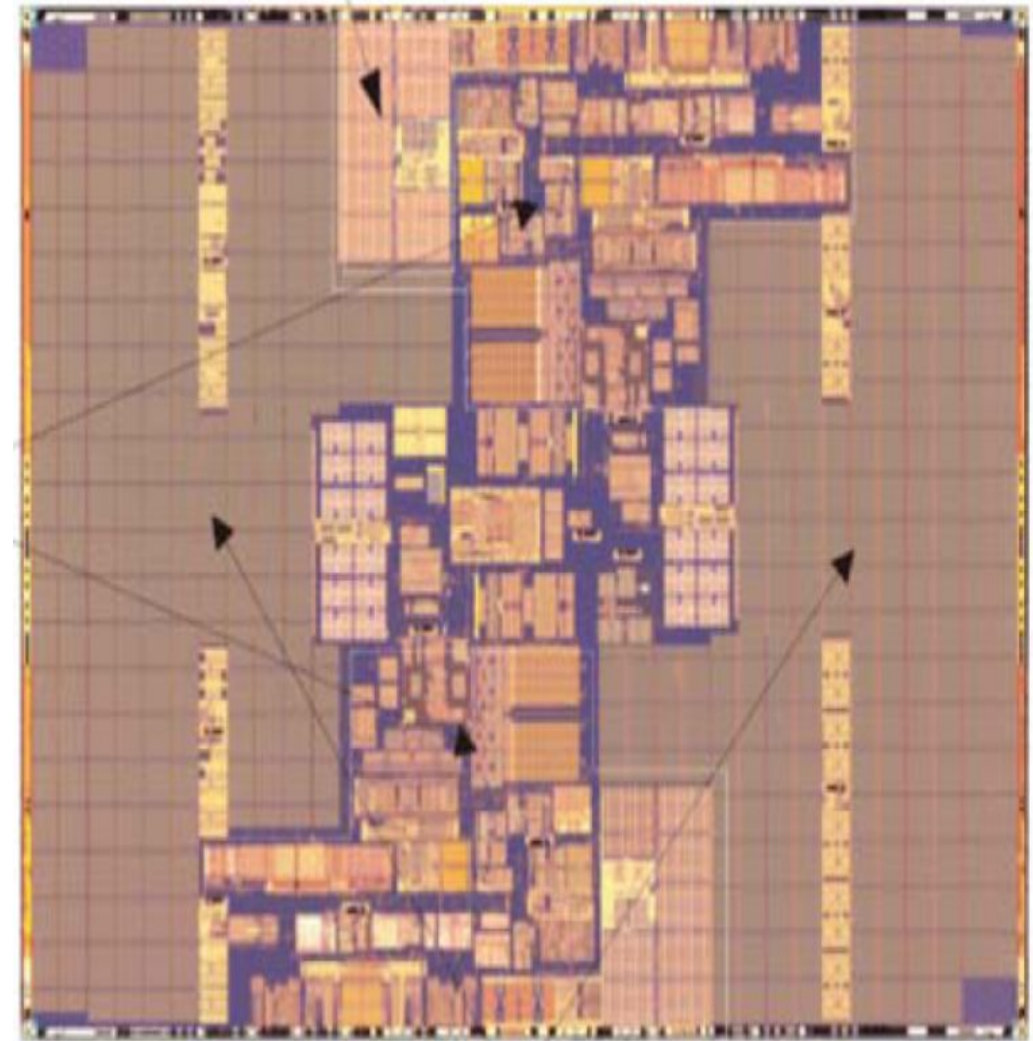
Changes in Computation

- Manual
- Mechanical
 - gears, chains, pulleys, and steam power
 - Punch cards
- Electro-mechanical
 - switches, relays
- Electrical
 - plugboards, vacuum tubes
 - later came DRUM memory, core memory, transistors and so on ...



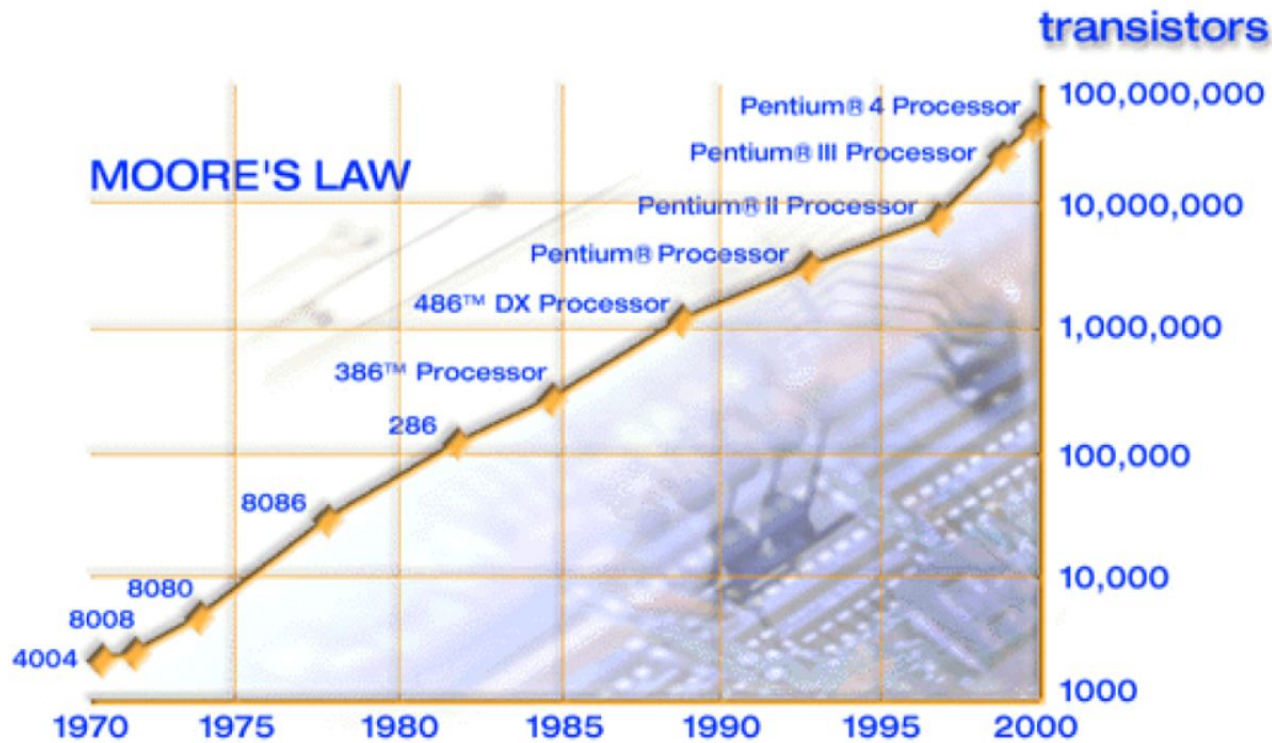
Computation on 2004

- 64-bit Itanium processor developed by Intel
- 1.7 billion transistors
- 1.7 GHz, issue up to 8 instructions per cycle
- 26 MByte of cache
- In ~30 years, about 100,000 fold growth in transistor count and performance

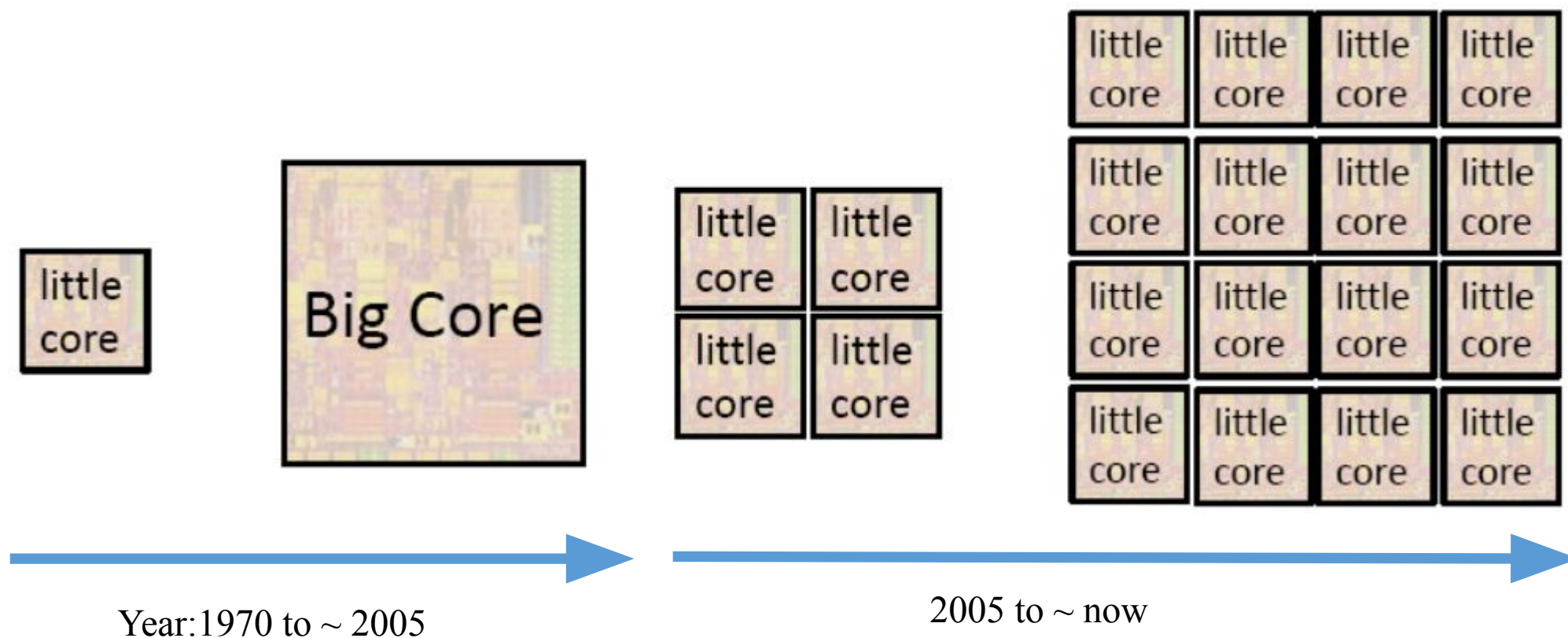


What Happened in Between

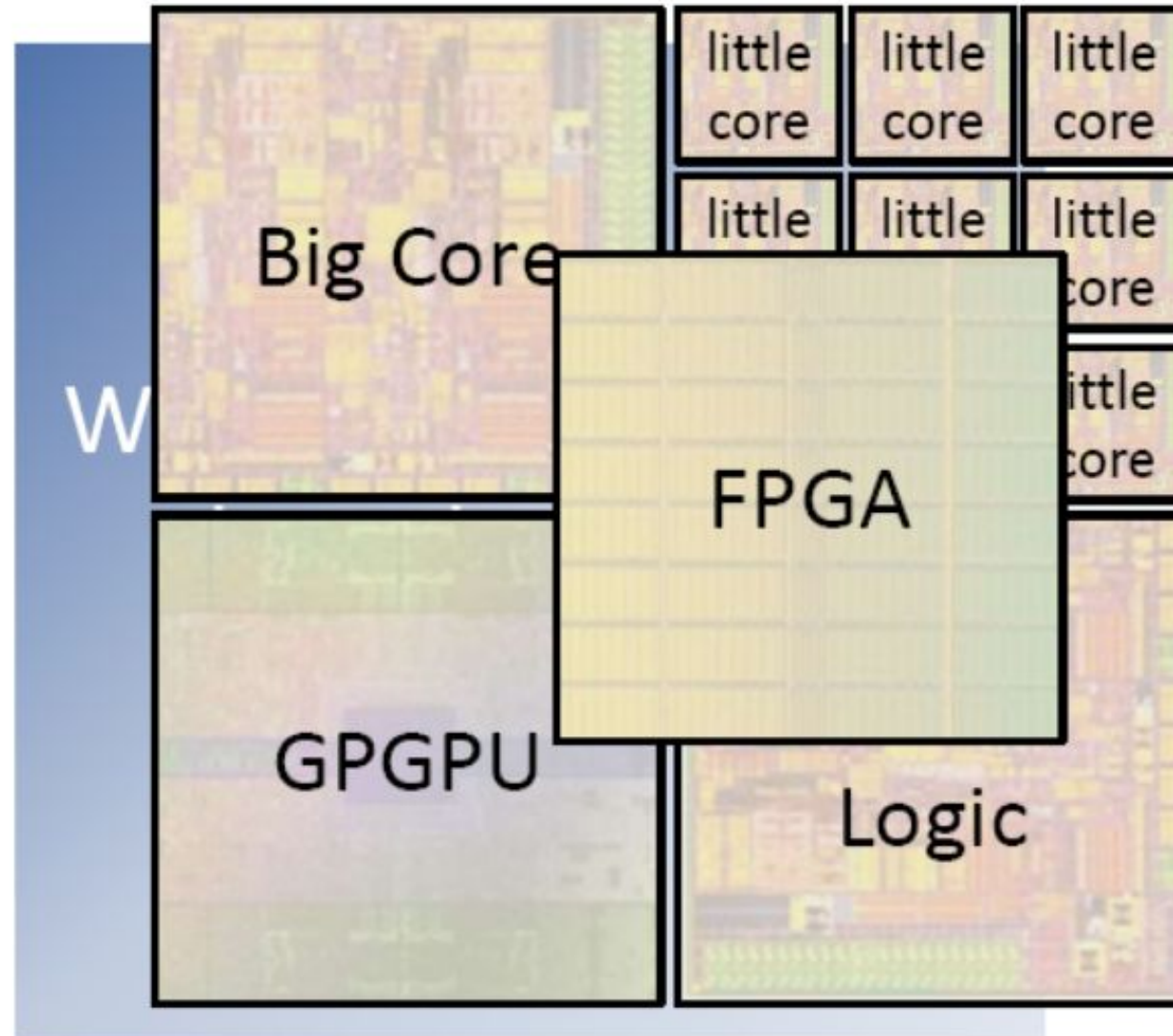
- **Moore's Law** refers to **Moore's** perception that the number of transistors on a microchip doubles every two years, though the cost of computers is halved.



Moore's Law Scaling with Cores



Future is about



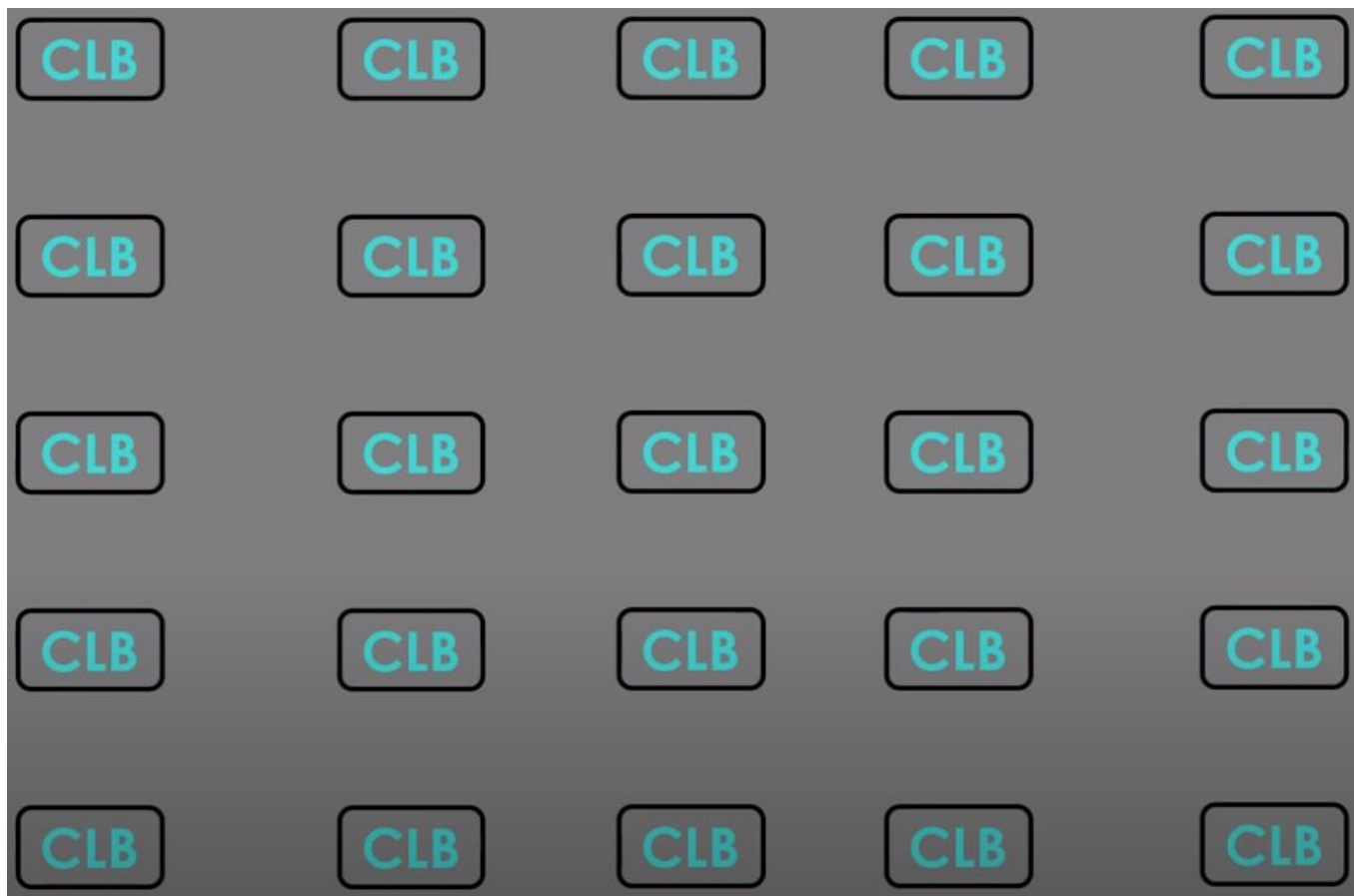
Quantum Computing

Carbon nanotube computer

- <https://www.nature.com/articles/nature12502>
- https://en.wikipedia.org/wiki/Carbon_nanotube_computer

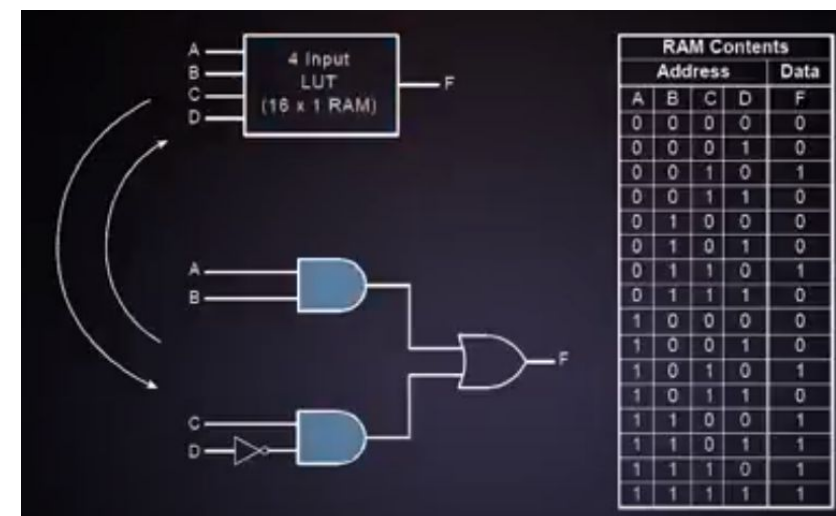
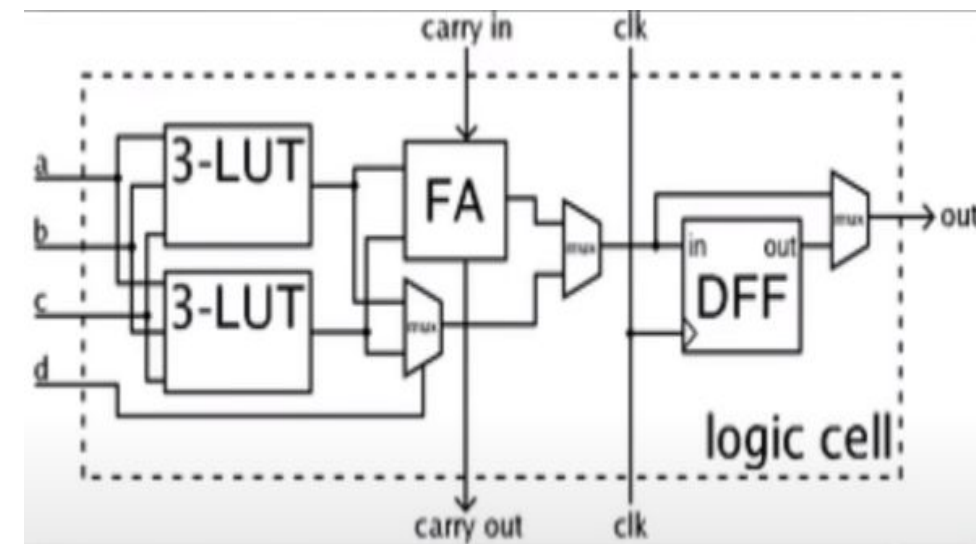
FPGA

- Field Programmable Gate Array



CLB: Configurable Logic block

Inside a CLB



4-inputs LUT

Quantum Computing

- <https://awards.acm.org/about/2020-acm-prize>
- <https://www.ibm.com/quantum-computing/>
- Google wants to build a **useful** quantum computer by 2029
 - <https://www.theverge.com/2021/5/19/22443453/google-quantum-computer-2029-decade-commercial-useful-qubits-quantum-transistor>
- Quantum Computing: Untangling the Hype (Talk at The Royal Institution)
 - <https://www.youtube.com/watch?v=wE1OCXvaDtc>

Homework

- Design a single-purpose processor for performing the only bubble sort.
- Design a general processor for a set of instructions:
 - ADD R1, R2, R3
 - SUB R1, R2, R3
 - LW R2, offset(R3)
 - SW R2, offset(R3)
 - BNE R1, R2, offset
- Has a instruction and a data memory
- Has 32 number of registers
- Register R0 holds zero value only.

Summary

- Motivation for automated Computation
- Dedicated processor Vs. General-purpose processor
- Limitation of Algorithm
- Building block of a program
- Steps to solve a problem by a computer/laptop
- Changes in Computation

ALU Operations

ALU control lines (3:0)	ALU Functions
0000	AND
0001	OR
0010	Add
0110	Subtract
0111	set on less than

Generation of Control: **ALU decoder** truth table

ALUOp	ALU control lines (2:0)	Funct
00	0010 (add)	X
01	0110 (subtract)	X
1X	0010 (add)	100000 (add)
1X	0110 (subtract)	100010 (sub)
1X	0000 (and)	100100 (and)
1X	0001 (or)	100101 (or)
1X	0111 (set less than)	101010 (slt)

Generation of Controls

Inst. opcode	ALUOp	Instr. operation	Funct field	Desired ALU action	ALUControl
100010 (LW)	00	load word	xxxxxx	add	0010
100011 (SW)	00	store word	xxxxxx	add	0010
000100 (BEQ)	01	branch equal	xxxxxx	subtract	0110
000000 (R-type)	10	add	100000	add	0010
R-type	10	Subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111
001000 (addi)	00	Immediate	xxxxxx	add	xxxx
000010 (j)	xx	jump	xxxxxx	jump	xxxx

